

- 5.1 Managing Disk Space: df, du, find command-Locating files dd, Command-Copying Disks, disk management-RAID.
 - 5.2 Backups: Need of backup, cpio & tar commands.
 - 5.3 Linux commands for disk management, fdisk, pydf, parted, lsblk, sfdisk, cfdisk.
 - 5.4 Creation and deletion of partition in linux, formatting partition :mkfs
-

5.1 Managing Disk Space: df, du:

There are many ways by which we can check the Linux system disk space. We can use a third-party app which displays the available disk space. Another way is the command-line by the Linux Terminal, some of which are df and du, where,

du means disk space used and
df means disk space free.

Check Disk Space using the df Command:

When executing this command, you will see the default columns:

Filesystem, Size, Used, Available, Use%, and Mounted On. It should look something like this:

```
tautvydas@support-2:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            7,8G     0   7,8G   0% /dev
tmpfs           1,6G    10M   1,6G   1% /run
/dev/nvme0n1p1  219G   5,6G  202G   3% /
tmpfs           7,8G   580M   7,3G   8% /dev/shm
tmpfs           5,0M    4,0K   5,0M   1% /run/lock
tmpfs           7,8G     0   7,8G   0% /sys/fs/cgroup
192.168.168.100:/home 100G   74G   27G  74% /home
tmpfs           1,6G    4,0K   1,6G   1% /run/user/0
tmpfs           1,6G   148K   1,6G   1% /run/user/5012
```

- **Filesystem** — provides the name of the file system.
- **Size** — gives us the total size of the specific file system.
- **Used** — shows how much disk space is used in the particular file system.
- **Available** — shows how much space is left in the file system.
- **Use%** — displays the percentage of disk space that is used.
- **Mounted On** — tells us the mount point of a particular file system.

By adding a certain option to the **df** command, you can check the disk space in Linux more precisely. These are the most popular options:

- **df -h** — it will display the result in a **human-readable** format.
- **df -m** — this command line is used to display information of file system usage in **MB**.
- **df -k** — to display file system usage in **KB**.
- **df -T** — this option will show the file system **type** (a new column will appear).
- **df -ht /home** — it allows you to view information about a specific file system in a readable format (in this case **/home** file system).
- **df --help** — it lists down other useful options that you can use, complete with their descriptions.

Check Disk Usage using the du Command:

Another important command is **du**, short for **Disk Usage**. It will show you details about the disk usage of files and directories on a Linux computer or server. With the **du** command, you need to specify which folder or file you want to check.

syntax:

```
du <options> <location of directory or file>
```

Let's take a look at real-world use of the **du** command with the Desktop directory:

- **du /home/user/Desktop** — this command line allows users to see into the disk usage of their Desktop folders and files (subdirectories are included as well).
- **du -h /home/user/Desktop** — just like with **df**, the option **-h** displays information in a **human-readable** format.
- **du -sh /home/user/Desktop** — the **-s** option will give us the total size of a specified folder (Desktop in this case).
- **du -m /home/user/Desktop** — the **-m** option provides us with folder and file sizes in **Megabytes** (we can use **-k** to see the information in **Kilobytes**).
- **du -h --time /home/user/Desktop** — this informs the last modification date of the displayed folders and files.
- **du --help** — it displays a list of available options and what they can be used for.

Combining Commands and Cleaning Disk Space:

You can get more information done by combining **df** and **du** command with other arguments. By doing this, you will get a better idea of which files you can delete to free up disk space. Just remember to start with **df** command to see which file system needs a cleanup the most. After that, you can proceed with these combinations.

1. Sorting Files By Size:

First, we gather files and folders on the Desktop in a readable format using the **du** command. Then, we pipe the result to the **sort** command together with the **-rn** option. The script will sort all the files and folders from largest to smallest to check the disk space use in Linux. The combination should look like this:

```
du -h /home/user/Desktop | sort -rn
```

Remember that you shouldn't necessarily delete files just because they are large. If you're not cautious, you might delete essential files that would break your project.

2. Excluding by File Size:

Let's say you want to see all files that are above a certain size. The most effective way to do that is by using the command below:

```
du -h /home/user/Desktop | grep '^s*[0-9\.]\|+G'
```

The **grep** command allows us to search for files based on a specified pattern. In this example, the script will return with any files bigger than 1 GB. If you want to single out 1 MB+ data, you can replace **G** with **M**.

3. Excluding File Types:

The last combination is useful when you need to exclude a particular file format from the search results.

For instance:

```
du -h /home/user/Desktop/ --exclude="*.txt"
```

The **--exclude=*.txt** argument makes sure the **du** command will display all file formats except for **.txt** documents.

The find command:

The find command in UNIX is a command line utility for walking a file hierarchy. It can be used to find files and directories and perform subsequent operations on them. It supports searching by file, folder, name, creation date, modification date, owner and permissions. By using the `_exec` other UNIX commands can be executed on files or folders found.

Syntax :

```
$ find [where to start searching from]
[expression determines what to find] [-options] [what to find]
```

Options :

- **-exec CMD:** The file being searched which meets the above criteria and returns 0 for as its exit status for successful command execution.
- **-ok CMD :** It works same as -exec except the user is prompted first.
- **-inum N :** Search for files with inode number `_N`.
- **-links N :** Search for files with `_N` links.
- **-name demo :** Search for files that are specified by `_demo`.
- **-newer file :** Search for files that were modified/created after `_file`.
- **-perm octal :** Search for the file if permission is `_octal`.
- **-print :** Display the path name of the files found by using the rest of the criteria.
- **-empty :** Search for empty files and directories.
- **-size +N/-N :** Search for files of `_N` blocks; `_N` followed by `_c` can be used to measure size in characters; `_+N` means size > `_N` blocks and `_-N` means size < 'N' blocks.
- **-user name :** Search for files owned by user name or ID `_name`.
- **\(expr \) :** True if `_expr` is true; used for grouping criteria combined with OR or AND.
- **! expr :** True if `_expr` is false.

1. Search a file with specific name.

```
$ find ./GFG -name sample.txt
```

It will search for sample.txt in GFG directory.

2. Search a file with pattern.

```
$ find ./GFG -name *.txt
```

It will give all files which have `_txt` at the end.

3. How to find and delete a file with confirmation.

```
$ find ./GFG -name sample.txt -exec rm -i {} \;
```

When this command is entered, a prompt will come for confirmation, if you want to delete sample.txt or not. if you enter `_Y/y_` it will delete the file.

4. Search for empty files and directories.

```
$ find ./GFG -empty
```

This command find all empty folders and files in the entered directory or sub-directories.

5. Search for file with entered permissions.

```
$ find ./GFG -perm 664
```

This command find all the files in the GFG directory or sub-directory with the given permissions.

6. Search text within multiple files.

```
$ find ./ -type f -name "*.txt" -exec grep 'Geek' {} \;
```

This command print lines which have `_Geek_` in them and `_-type f_` specifies the input type is a file.

Locating files:

The **locate** command is the first resort that a Linux user should utilize because it's much faster than anything else out there. The reason for this unmatched speed is that the locate command isn't actually searching your local hard disks for the files or directories that you need to find, but more like reads through the **mlocate.db** database file which contains all file paths in your system.

If you have not installed locate yet, use these commands:

For Debian and Ubuntu

```
$sudo apt-get install locate
```

Prepare locate command for first use

To update the **mlocate.db** database before first use, run:

```
$sudo updatedb
```

To use locate, open a terminal and type locate followed by the file name you are looking for. In this example, I'm searching for files that contain the word 'sunny' in their name.

```
$locate sunny
```

Locate can also tell you how many times a search keyword is matched in the database. This is achieved by including a **--c** parameter in the command, which stands for **-count**.

```
$locate -c sunny
```

What users need to have in mind here is that **locate** needs the specific and exact name of the file you're looking for, contrary to other more flexible tools. That means that locate is great for finding something that you're sure about its name, but not so great for when you don't exactly remember the file name.

```
$locate howtoforge
```

vs

```
$locate Howtoforge
```

Moreover, and since **—locatell** reads a database file, the results may be outdated and not completely true. To mitigate this issue, you can update your file paths database by typing **—sudo updatedb** on a terminal session before the use of locate command.

The **_dd** command in Linux:

dd is a command-line utility for Unix and Unix-like operating systems whose primary purpose is to convert and copy files. **dd** stands for Data Description

- On Unix, device drivers for hardware (such as hard disk drives) and special device files (such as `/dev/zero` and `/dev/random`) appear in the file system just like normal files.
- **dd** can also read and/or write from/to these files, provided that function is implemented in their respective drivers
- As a result, **dd** can be used for tasks such as backing up the boot sector of a hard drive, and obtaining a fixed amount of random data.
- The **dd** program can also perform conversions on the data as it is copied, including byte order swapping and conversion to and from the ASCII and EBCDIC text encodings.

Usage : The command line syntax of **dd** differs from many other Unix programs, in that it uses the syntax *option=value* for its command line options, rather than the more-standard *-option value* or *—option=value* formats. By default, **dd** reads from stdin and writes to stdout, but these can be changed by using the **if** (input file) and **of** (output file) options.

Some practical examples on **dd** command :

1. **To backup the entire harddisk :** To backup an entire copy of a hard disk to another hard disk connected to the same system, execute the **dd** command as shown. In this **dd** command example, the UNIX device name of the source hard disk is `/dev/hda`, and device name of the target hard disk is `/dev/hdb`.

#dd if = /dev/sda of = /dev/sdb

where,

- **if=/dev/sda** : Input disk (source)

- **of=/dev/sdb** : Output disk (destination)

- “if” represents inputfile, and “of” represents output file. So the exact copy of /dev/sda will be available in /dev/sdb.
- If there are any errors, the above command will fail. If you give the parameter “conv=noerror” then it will continue to copy if there are read errors.
- Input file and output file should be mentioned very carefully. Just in case, you mention source device in the target and vice versa, you might loss all your data.
- To copy, hard drive to hard drive using dd command given below, sync option allows you to copy everything using synchronized I/O.
- **# dd if = /dev/sda of = /dev/sdb conv=noerror, sync**

2. **To backup a Partition** : You can use the device name of a partition in the input file, and in the output either you can specify your target path or image file as shown in the dd command.

dd if=/dev/hda1 of=~/partition.img

3. **To create an image of a Hard Disk** : Instead of taking a backup of the hard disk, you can create an image file of the hard disk and save it in other storage devices. There are many advantages of backing up your data to a disk image, one being the ease of use. This method is typically faster than other types of backups, enabling you to quickly restore data following an unexpected catastrophe. It creates the image of a hard disk /dev/hda.

dd if = /dev/hda of = ~/hdadisk.img

4. **To restore using the Hard Disk Image** : To restore a hard disk with the image file of an another hard disk, the following dd command can be used

dd if = hdadisk.img of = /dev/hdb

The image file hdadisk.img file, is the image of a /dev/hda, so the above command will restore the image of /dev/hda to /dev/hdb.

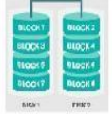
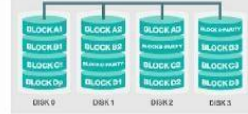
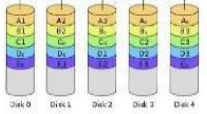
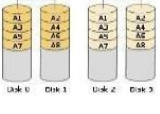
5. **To create CDROM Backup** : dd command allows you to create an iso file from a source file. So we can insert the CD and enter dd command to create an iso file of a CD content.

dd if = /dev/cdrom of = tgsservice.iso bs = 2048

dd command reads one block of input and process it and writes it into an output file. You can specify the block size for input and output file. In the above dd command example, the parameter “*bs*” specifies the block size for the both the input and output file. So dd uses **2048bytes** as a block size in the above command.

Disk Management-RAID

RAID is a Redundant Array of Inexpensive disks, but nowadays it is called Redundant Array of Independent drives. Earlier it is used to be very costly to buy even a smaller size of disk, but nowadays we can buy a large size of disk with the same amount like before. Raid is just a collection of disks in a pool to become a logical volume.

S.No	PROPERTIES	RAID 0	RAID 1	RAID 5	RAID 6	RAID 10
1.	PRINCIPLE	Striping	Mirroring	Striping with parity	Striping with double parity	Mirroring and Striping
2.	MINIMUM DISK REQUIRED	Two (2)	Two (2)	Three (two for data one for parity)	Four (2 for data 2 for parity)	Four
3.	FAULT TOLERANCE	None	n-1 drive	One	Two	One drive in each subarray
4.	BENEFITS	1. Great performance, 2. All storage capacity used, 3. Easy to implemt	1. excellent speed 2. very simple 3. if a drive fails just copy data of one to another.	1. Read data transactions very fast 2. if a drive fails storage controllers rebuild data using parity.	1. Read data transactions very fast 2. if two drive fails storage controllers rebuild data on replacement drive. 3. More secure than RAID 5	1.combines advantage of RAID0 And RAID 1 in one system. 2. provides data security by mirroring and speed up by striping.
5.	DISADVANTAGES	No fault tolerant	Effective storage capacity becomes half.	1.Complex technology. 2.Data restoration takes too much time.If 2 nd drive fails then data is completely lost	1.Write data transaction slower than RAID 5 2. very complex technology.	1.Lower usable capacity 2. High Cost 3. Limited scalability
6.	FIGURE					
7.	APPLICATIONS	Where data is not critical eg postproduction, CAD.	Accounting, gaming , video editing etc	Fileservers, storage servers, strimming data etc	Fileserver, general storage servers, backup servers.	Database servers.

Understanding RAID Setups in Linux

Raid contains groups or sets or Arrays. A combine of drivers make a group of disks to form a RAID Array or RAID set. It can be a minimum of 2 number of disk connected to a raid controller and make a logical volume or more drives can be in a group. Only one Raid level can be applied in a group of disks. Raid are used when we need excellent performance. According to our selected raid level, performance will differ. Saving our data by fault tolerance & high availability.

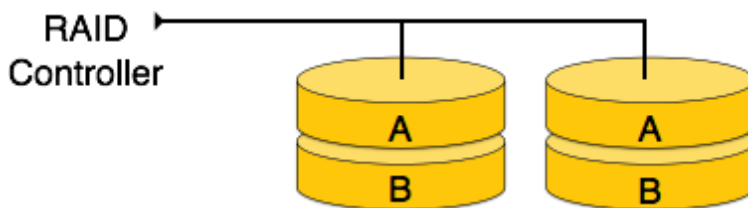
RAID 0

In this level, a striped array of disks is implemented. The data is broken down into blocks and the blocks are distributed among disks. Each disk receives a block of data to write/read in parallel. It enhances the speed and performance of the storage device. There is no parity and backup in Level 0.



RAID 1

RAID 1 uses mirroring techniques. When data is sent to a RAID controller, it sends a copy of data to all the disks in the array. RAID level 1 is also called **mirroring** and provides 100% redundancy in case of a failure.



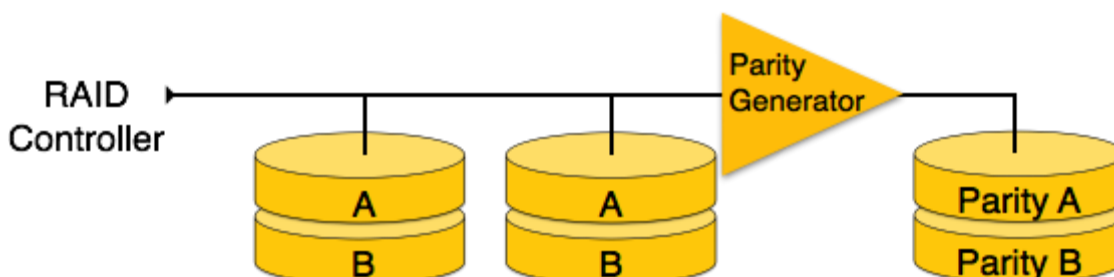
RAID 2

RAID 2 records **Error Correction Code using Hamming distance for its data**, striped on different disks. Like level 0, each data bit in a word is recorded on a separate disk and ECC codes of the data words are stored on a different set disks. **Due to its complex structure and high cost, RAID 2 is not commercially available.**



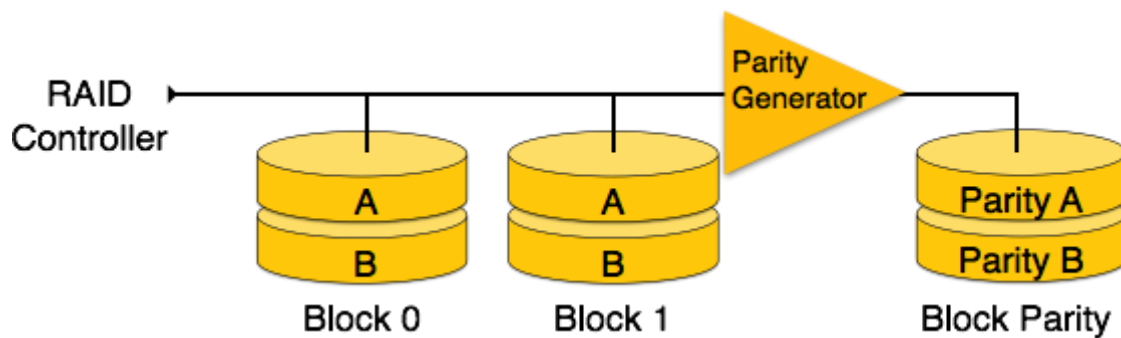
RAID 3

RAID 3 stripes the data onto multiple disks. **The parity bit generated for data word is stored on a different disk.** **This technique makes it to overcome single disk failures.**



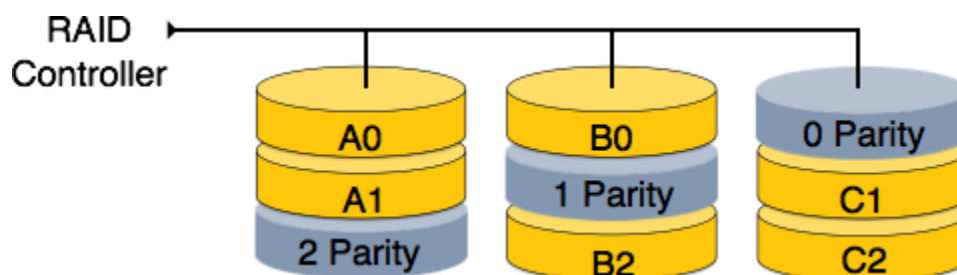
RAID 4

In this level, an entire block of data is written onto data disks and then the parity is generated and stored on a different disk. **Note that level 3 uses byte-level striping, whereas level 4 uses block-level striping.** **Both level 3 and level 4 require at least three disks to implement RAID.**



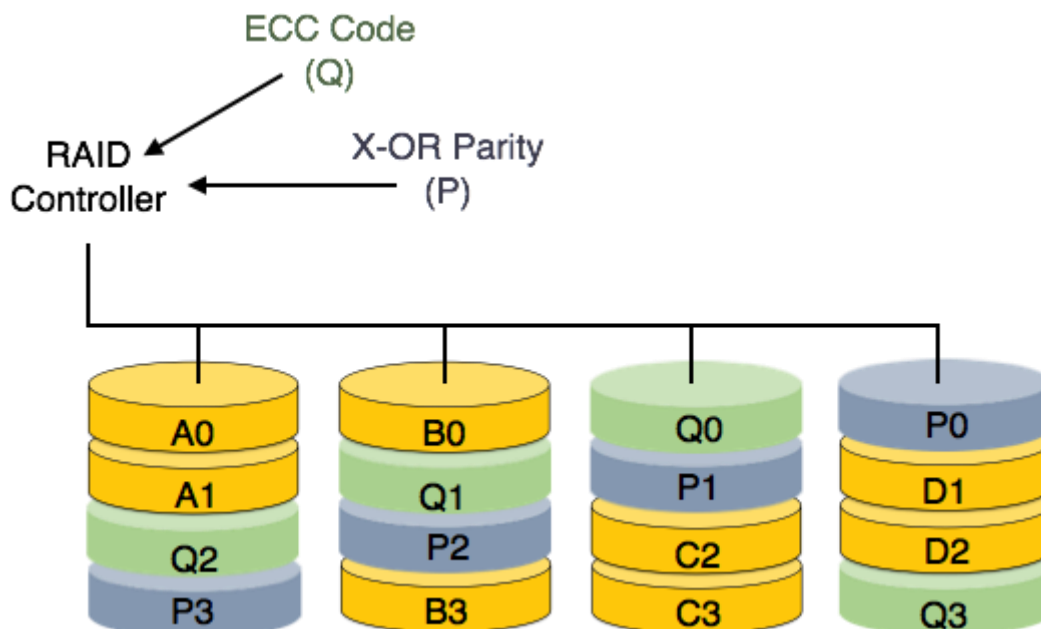
RAID 5

RAID 5 writes whole data blocks onto different disks, but the parity bits generated for data block stripe are distributed among all the data disks rather than storing them on a different dedicated disk.



RAID 6

RAID 6 is an extension of level 5. In this level, two independent parities are generated and stored in distributed fashion among multiple disks. Two parities provide additional fault tolerance. This level requires at least four disk drives to implement RAID.



Featured Concepts of RAID

1. **Parity** method in raid regenerate the lost content from parity saved information's. RAID 5, RAID 6 Based on Parity.
2. **Stripe** is sharing data randomly to multiple disk. This won't have full data in a single disk. If we use 3 disks half of our data will be in each disks.
3. **Mirroring** is used in RAID 1 and RAID 10. Mirroring is making a copy of same data. In RAID 1 it will save the same content to the other disk too.
4. **Hot spare** is just a spare drive in our server which can automatically replace the failed drives. If any one of the drive failed in our array this hot spare drive will be used and rebuild automatically.
5. **Chunks** are just a size of data which can be minimum from 4KB and more. By defining chunk size we can increase the I/O performance.

RAID's are in various Levels. Here we will see only the RAID Levels which is used mostly in real environment.

1. **RAID0 = Striping**
2. **RAID1 = Mirroring**
3. **RAID5 = Single Disk Distributed Parity**
4. **RAID6 = Double Disk Distributed Parity**
5. **RAID10 = Combine of Mirror & Stripe. (Nested RAID)**

RAID are managed using **mdadm** package in most of the Linux distributions.

Here we will discuss about **RAID 1** which is also known as **disk mirroring**. RAID 1 creates identical copies of data. If you have two hard drives in RAID 1, then data will be written to both drives. The two hard drives have the same data.

The nice part about RAID 1 is that if one of your hard drive fails, your computer or server would still be up and running because you have a complete, intact copy of the data on the other hard drive. You can pull the failed hard drive out while the computer is running, insert a new hard drive and it will automatically rebuilds the mirror.

The downside of RAID 1 is that you don't get any extra disk space. If your two hard drives are both 1TB, then the total usable volume is 1TB instead of 2TB.

Software RAID and Hardware RAID:

Software RAID have low performance, because of consuming resource from hosts. Raid software need to load for read data from software raid volumes. Before loading raid software, OS need to get boot to load the raid software. No need of Physical hardware in software raids. Zero cost investment.

Hardware RAID have high performance. They are dedicated RAID Controller which is Physically built using PCI express cards. It won't use the host resource. They have NVRAM for cache to read and write. Stores cache while rebuild even if there is power-failure, it will store the cache using battery power backups. Very costly investments needed for a large scale.

Hardware RAID Card will look like below:



Hardware RAID

Hardware RAID vs Software RAID

To set up RAID, you can either use a hard drive controller, or use a piece of software to create it. A hard drive controller is a PCIe card that you put into a computer. Then you connect your hard drives to this card. When you boot up the computer, you are going to see an option that allows you to configure the RAID. You can install an operating system on top of hardware RAID which can increase uptime.

Software RAID requires you already installed an operating system. It's good for storing data.

5.2 Backups: Need of backup, cpio & tar commands.

Need of backup:

1. Data Loss Prevention

We've all heard about or experienced a tragic loss of data. The main reason for data backup is to save important files if a system crash or hard drive failure occurs.

2. Operation Plan B

There should be additional data backups if the original backups result in data corruption or hard drive failure. This option is best done via the cloud or offsite storage. Additional backups are necessary if natural or man-made disasters occur. Storms and warfare can lead to the destruction of servers and computers due to fires and floods. Luckily, we are in the age of cloud technology, where backup your data has become easier and more secure than ever before.

3. Tax Reporting and Audits

Tax authorities are notorious for audits. Laws differ among countries, but it is important for companies to save financial and accounting data for tax reporting purposes. With data backup, companies can save face during audits.

4. Client Relationship

Saved information improves client relationship management, which leads to increased marketing and sales. Additionally, saved client information builds trust and value of a company.

5. Investor Relations

Per investor relations, data backup reduces the tedious time to compile annual reports to shareholders. Saved information symbolizes a company's due diligence and organization. Without data backup, shareholders cannot make informed decisions or determine a company's value.

6. Archiving

Backed up information streamlines the development of archives. With digital information, company history is in the making.

7. Competitive Gain

Saved company data can be a competitive advantage because there are many businesses that fail to data backup important information.

8. Improved Productivity

With existing backed up files, companies improve productivity by reducing wasted time. Archived files lead to comparative studies of the past and present to devise a more effective plan.

9. No Wasted Time

Data backup reduces _wasted time_ by preventing repetitions. Thus, employees do not have to rewrite reports.

10. Peace of Mind

Regular data backups lead to peace of mind. In the event, a cybercrime, system crashes or disasters occur, there is a backup ready to go to restart a company's archive. It is never too late to start saving important company data. In the end, data backup is necessary to save the business from losing investors and customers and closing down.

The cpio command:

cpio stands for **-copy in, copy out-**. It is used for processing the archive files like **.cpio* or **.tar*. This command can copy files to and from archives.

1. Copy-out Mode: Copy files named in name-list to the archive

Syntax:

```
cpio -o < name-list > archive
```

2. Copy-in Mode: Extract files from the archive

Syntax:

```
cpio -i < archive
```

3. Copy-pass Mode: Copy files named in name-list to destination-directory

Syntax:

- `cpio -p destination-directory < name-list`

Policy Options:

- **-i, -extract:** Extract files from an archive and it runs only in copy-in mode.
- **-o, -create:** Create the archive and it runs only in copy-out mode.
- **-p, -pass-through:** Run in copy-pass mode.
- **-t, -list:** Print a table of contents of all the inputs present.

Operation modifiers valid in any Mode:

- **-B:** Changes the I/O block size to 5120 bytes.
- **-c:** Use the old portable (ASCII) archive format.
- **-C, -io-size=NUMBER:** Set the I/O block size to the given particular NUMBER of bytes.
- **-D, -directory=DIR:** Changes to Directory *DIR*.
- **-H, -format=FORMAT:** Use given arc.
- **-v, -verbose:** List the files processed in a particular task.
- **-V, -dot:** Print `-.||` for each file processed in a particular task.
- **-W, -warning=FLAG:** Control warning display. Currently FLAG is one of `__none__`, `__truncate__`, `__all__`.

Examples:

1. To create a *.cpio file : We can create *.cpio files containing files and directory with the help of cpio command.

Syntax:

`cpio -ov < name-list > archive`

Here `-ov` is used as `-o` create the new archive and `-v` list the files processed.

```
linux@ubuntu:~/files$ ls
file  file2
linux@ubuntu:~/files$ ls | cpio -ov > /home/linux/compress.cpio
file
file2
1 block
linux@ubuntu:~/files$
```

2. To extract a *.cpio file: We can extract *.cpio files containing files and directory with the help of cpio command.

Syntax:

`cpio -iv < archive`

```
linux@ubuntu:~/files$ ls
linux@ubuntu:~/files$ cpio -iv < /home/linux/compress.cpio
file
file2
1 block
linux@ubuntu:~/files$
```

3.To create *.tar archive file using cpio: The cpio helps to create a *.tar archive.

Syntax:

`cpio -ov -H tar > archive`

```
linux@ubuntu:~/files$ ls
file  file2
linux@ubuntu:~/files$ ls | cpio -ov -H tar > /home/linux/txtfile.tar
file
file2
4 blocks
linux@ubuntu:~/files$
```

- 4. To extract *.tar archive file using cpio:** The cpio helps to extract *.tar files containing files and directory.

Syntax:

cpio -iv -F < archive

```
linux@ubuntu:~/files$ ls
linux@ubuntu:~/files$ cpio -iv -F /home/linux/txtfile.tar
file
file2
3 blocks
linux@ubuntu:~/files$
```

- 5. To create a *.cpio archive with selected files:** We can create *.cpio files containing specific files with the help of cpio command. In the example we are using .txt files.

Syntax:

find . -iname "*.txt" | cpio -ov > archive

```
linux@ubuntu:~/files$ ls
file2.txt  file.txt
linux@ubuntu:~/files$ find . -iname "*.txt" | cpio -ov > /home/linux/txtfiles.cpio
./file2.txt
./file.txt
1 block
linux@ubuntu:~/files$
```

- 6. To create a *.tar archive with selected files:** We can create *.tar files containing specific files with the help of cpio command. In the example we are using .txt files.

Syntax:

find . -iname "*.txt" | cpio -ov -H tar > archive

```
linux@ubuntu:~/files$ find . -iname "*.txt" | cpio -ov -H tar > /home/linux/txtfile.tar
./file2.txt
./file.txt
4 blocks
linux@ubuntu:~/files$
```

7. **To only view *.tar archive file using cpio:** The cpio helps to view *.tar files containing files and directory.

Syntax:

cpio -it -F < archive

```
linux@ubuntu:~/files$ ls
linux@ubuntu:~/files$ cpio -it -F /home/linux/txtfile.tar
file2.txt
file.txt
3 blocks
linux@ubuntu:~/files$
```

The tar command:

The Linux `_tar` stands for **tape archive**, is used to create Archive and extract the Archive files. tar command in Linux is one of the important command which provides archiving functionality in Linux. We can use Linux tar command to create compressed or uncompressed Archive files and also maintain and modify them.

Syntax:

tar [options] [archive-file] [file or directory to be archived]

Options:

- c : Creates Archive
- x : Extract the archive
- f : creates archive with given filename
- t : displays or lists files in archived file
- u : archives and adds to an existing archive file
- v : Displays Verbose Information
- A : Concatenates the archive files
- z : zip, tells tar command that create tar file using gzip
- j : filter archive tar file using tbzip
- W : Verify a archive file
- r : update or add file or directory in already existed .tar file

What is an Archive file?

An Archive file is a file that is composed of one or more files along with metadata. Archive files are used to collect multiple data files together into a single file for easier portability and storage, or simply to compress files to use less storage space.

Examples:

1. Creating an uncompressed tar Archive using option -cvf : This command creates a tar file called file.tar which is the Archive of all .c files in current directory.

```
$ tar cvf file.tar *.c
```

Output :

os2.c

os3.c

os4.c

2. Extracting files from Archive using option -xvf : This command extracts files from Archives.

```
$ tar xvf file.tar
```

Output :

```
os2.c
```

```
os3.c
```

```
os4.c
```

3. gzip compression on the tar Archive, using option -z : This command creates a tar file called file.tar.gz which is the Archive of .c files.

```
$ tar cvzf file.tar.gz *.c
```

4. Extracting a gzip tar Archive *.tar.gz using option -xvzf : This command extracts files from tar archived file.tar.gz files.

```
$ tar xvzf file.tar.gz
```

5. Creating compressed tar archive file in Linux using option -j : This command compresses and creates archive file less than the size of the gzip. Both compress and decompress takes more time then gzip.

```
$ tar cvfj file.tar.tbz example.cpp
```

Output :

```
$tar cvfj file.tar.tbz example.cpp
```

```
example.cpp
```

```
$tar tvf file.tar.tbz
```

```
-rwxrwxrwx root/root    94 2017-09-17 02:47 example.cpp
```

6. Untar single tar file or specified directory in Linux : This command will Untar a file in current directory or in a specified directory using -C option.

```
$ tar xvfj file.tar
```

or

```
$ tar xvfj file.tar -C path of file in directoy
```

7. Untar multiple .tar, .tar.gz, .tar.tbz file in Linux : This command will extract or untar multiple files from the tar, tar.gz and tar.bz2 archive file. For example the above command will extract -fileA|| -fileB|| from the archive files.

```
$ tar xvf file.tar "fileA" "fileB"
```

or

```
$ tar zxvf file1.tar.gz "fileA" "fileB"
```

or

```
$ tar jxvf file2.tar.tbz "fileA" "fileB"
```

8. Check size of existing tar, tar.gz, tar.tbz file in Linux : The above command will display the size of archive file in Kilobytes(KB).

```
$ tar czf file.tar | wc -c
```

or

```
$ tar czf file1.tar.gz | wc -c
```

or

```
$ tar czf file2.tar.tbz | wc -c
```

9. Update existing tar file in Linux

```
$ tar rvf file.tar *.c
```

Output :

```
os1.c
```

10. list the contents and specify the tarfile using option -tf : This command will list the entire list of archived file. We can also list for specific content in a tarfile

```
$ tar tf file.tar
```

Output :

```
example.cpp
```

11. Applying pipe to through _grep command' to find what we are looking for : This command will list only for the mentioned text or image in grep from archived file.

```
$ tar tvf file.tar | grep "text to find"
```

or

```
$ tar tvf file.tar | grep "filename.file extension"
```

12. We can pass a file name as an argument to search a tarfile : This command views the archived files along with their details.

```
$ tar tvf file.tar filename
```

13. Viewing the Archive using option -tvf

```
$ tar tvf file.tar
```

Output :

```
-rwxrwxrwx root/root    191 2017-09-17 02:20 os2.c
-rwxrwxrwx root/root    218 2017-09-17 02:20 os3.c
-rwxrwxrwx root/root    493 2017-09-17 02:20 os4.c
```

What are wildcards in Linux

Alternatively referred to as a `_wild character` or `_wildcard character`, a wildcard is a symbol used to replace or represent one or more characters. Wildcards are typically either an asterisk (*), which represents one or more characters or question mark (?), which represents a single character.

Example :

14. To search for an image in .png format : This will extract only files with the extension .png from the archive file.tar. The **--wildcards** option tells tar to interpret wildcards in the name of the files

to be extracted; the filename (*.png) is enclosed in single-quotes to protect the wildcard (*) from being expanded incorrectly by the shell.

```
$ tar tvf file.tar --wildcards '*.png'
```

Note: In above commands `| * |` is used in place of file name to take all the files present in that particular directory.

5.3 Linux commands for disk management, fdisk, pydf , parted, lsblk, sfdisk, cfdisk.

Linux disk management includes several important tasks such as adding or removing storage devices, creating and deleting partitions, mounting partitions on appropriate directories and making file system in partitions.

A. Linux commands for disk management: fdisk

fdisk also known as **format disk** is a dialog-driven command in Linux used for creating and manipulating disk partition table. It is used for the view, create, delete, change, resize, copy and move partitions on a hard drive using the dialog-driven interface.

fdisk allows you to create a maximum of four primary partitions and the number of logical partition depends on the size of the hard disk you are using. It allows the user:

- To Create space for new partitions.
- Organizing space for new drives.
- Re-organizing old drives.
- Copying or Moving data to new disks(partitions).

Syntax:

```
fdisk [options] device
```

or

```
fdisk -l [device...]
```

1. View All Disk Partitions:The first thing to do before doing any thing with the disks and partition is to view basic details about all available partition in the system. The below command is used to list the partitions on your system and see their `/dev` names. For example `/dev/sda`, `/dev/sdb` or `/dev/sdc`.

\$ sudo fdisk -l

```
linux@ubuntu:~$ sudo fdisk -l
Disk /dev/loop0: 140.9 MiB, 147722240 bytes, 288520 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes

Disk /dev/loop1: 140.7 MiB, 147496960 bytes, 288080 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes

Disk /dev/loop2: 14.5 MiB, 15196160 bytes, 29680 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes

Disk /dev/loop3: 3.7 MiB, 3887104 bytes, 7592 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes

Disk /dev/loop4: 13 MiB, 13619200 bytes, 26600 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes

Disk /dev/loop5: 34.7 MiB, 36323328 bytes, 70944 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes

Disk /dev/loop6: 91 MiB, 95416320 bytes, 186360 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes

Disk /dev/loop7: 34.8 MiB, 36503552 bytes, 71296 sectors
Units: sectors of 1 * 512 = 512 bytes
```

4. View Partition on a Specific Disk: Below command is used to view all disk partitions on device `/dev/sda`.

\$ sudo fdisk -l /dev/sda

```
linux@ubuntu:~$ sudo fdisk -l /dev/sda
Disk /dev/sda: 20 GiB, 21474836480 bytes, 41943040 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0xa4f02e0d

Device      Boot Start      End  Sectors  Size Id Type
/dev/sda1   *      2048 41940991 41938944   20G 83 Linux
linux@ubuntu:~$
```

3. View all fdisk Commands: To see all the command which are available under fdisk command you can use **/dev/sda** partition with fdisk command.

\$ sudo fdisk /dev/sda

```
linux@ubuntu:~$ sudo fdisk /dev/sda

Welcome to fdisk (util-linux 2.31.1).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.


Command (m for help):
```

Note: This will prompt for a command. Type **m** for seeing all the operations which can perform on **/dev/sda**. After pressing **m** you will get:

```
Command (m for help): n

Help:

DOS (MBR)
 a toggle a bootable flag
 b edit nested BSD disklabel
 c toggle the dos compatibility flag

Generic
 d delete a partition
 F list free unpartitioned space
 l list known partition types
 n add a new partition
 p print the partition table
 t change a partition type
 v verify the partition table
 i print information about a partition

Misc
 n print this menu
 u change display/entry units
 x extra functionality (experts only)

Script
 I load disk layout from sfdisk script file
 O dump disk layout to sfdisk script file

Save & Exit
 w write table to disk and exit
 q quit without saving changes

Create a new label
 g create a new empty GPT partition table
 G create a new empty SGI (IRIX) partition table
 o create a new empty DOS partition table
 s create a new empty Sun partition table

Command (m for help):
```

4. Create a Hard Disk Partition: For this go inside the hard drive partition that is the **/dev/sda** partition, and use the following command:

\$ sudo fdisk /dev/sda

Now you have to type **n** to create new partition and then type **p** for making a primary partition and **e** for making an extended or logical partition depending on the type of partition to make.

```
Command (m for help): n
Partition type
   p   primary (0 primary, 0 extended, 4 free)
   e   extended (container for logical partitions)
Select (default p): e
Partition number (1-4, default 1): 2
First sector (2048-41938943, default 2048):
Last sector, +sectors or +size[K,M,G,T,P] (2048-41938943, default 41938943): 2048

Created a new partition 2 of type 'Extended' and of size 512 B.

Command (m for help):
```

Run **w** command to write the changes and reboot your system.

5. Delete a Hard Disk Partition: To delete a partition for the hard disk and free up space occupied by that partition for example **/dev/sdb**. Go to the command menu using following:

\$ sudo fdisk /dev/sda

and then type **d** to go to the delete partition menu. It will prompt the partition number you want to delete(type the number).

```
linux@ubuntu:~$ sudo fdisk /dev/sda1

Welcome to fdisk (util-linux 2.31.1).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.


Command (m for help): d
Partition number (1,5, default 5): 5

Partition 5 has been deleted.

Command (m for help):
```

Run **w** command to write the changes and reboot the system.

6. How to view the size of your Partition:

\$ sudo fdisk -s /dev/sda

```
linux@ubuntu:~$ sudo fdisk -s /dev/sda
20971520
linux@ubuntu:~$
```

Note:

1. To check for the manual page of fdisk command, use the following command:

\$ man fdisk

2. To see the help message and listing of all options, use the following command option:

\$ sudo fdisk -h

```
linux@ubuntu:~$ sudo fdisk -h
```

Usage:

```
fdisk [options] <disk>      change partition table
fdisk [options] -l [<disk>] list partition table(s)
```

Display or manipulate a disk partition table.

Options:

```
-b, --sector-size <size>      physical and logical sector size
-B, --protect-boot            don't erase bootbits when creating a new label
-c, --compatibility[=<mode>]   mode is 'dos' or 'nondos' (default)
-L, --color[=<when>]          colorize output (auto, always or never)
                               colors are enabled by default
-l, --list                    display partitions and exit
-o, --output <list>          output columns
-t, --type <type>            recognize specified partition table type only
-u, --units[=<unit>]         display units: 'cylinders' or 'sectors' (default)
-s, --getsz                   display device size in 512-byte sectors [DEPRECATED]
                               --bytes
                               print SIZE in bytes rather than in human readable format
-w, --wipe <mode>            wipe signatures (auto, always or never)
-W, --wipe-partitions <mode> wipe signatures from new partitions (auto, always or never)

-C, --cylinders <number>     specify the number of cylinders
-H, --heads <number>         specify the number of heads
-S, --sectors <number>       specify the number of sectors per track

-h, --help                    display this help
-V, --version                 display version
```

Available output columns:

```
gpt: Device Start End Sectors Size Type Type-UUID Attrs Name UUID
dos: Device Start End Sectors Cylinders Size Type Id Attrs Boot End-C/H/S Start-C/H/S
bsd: Slice Start End Sectors Cylinders Size Type Bsize Cpg Fsize
sgi: Device Start End Sectors Cylinders Size Type Id Attrs
sun: Device Start End Sectors Cylinders Size Type Id Flags
```

For more details see fdisk(8).

B. Linux commands for disk management: pydf

The `pydf` (Python Disk File System) is an advanced command line tool and a **good alternative to Linux `df` command**.

It is used to display the amount of used and available disk space on a Linux file systems, same like `df` command, but in different colours. The output of the `pydf` command can be customizable according to your needs.

This `pydf` command is written in **python** language that displays the amount of disk usage and available space on Linux mounted file system, using custom colours for different file system types.

Installing pydf

By default, `pydf` tool is not installed on Linux distributions, you need to install it using third party repository. So, first enable EPEL repository and afterwards enabling it, use the following `yum` command to install it.

On RHEL/CentOS/Fedora/Scientific Linux

```
# yum install pydf
```

Sample Output

```
Loaded plugins: fastestmirror, security
Determining fastest mirrors
epel/metalink | 4.2 kB
  * base: mirror.nbrc.ac.in
  * epel: mirrors.ispros.com.bd
  * extras: mirror.nbrc.ac.in
base | 3.7 kB
epel | 3.9 kB
epel/primary_db | 4.2 MB
extras | 3.5 kB
updates | 3.5 kB
updates/primary_db | 1.9 MB
Setting up Install Process
Resolving Dependencies
--> Running transaction check
---> Package pydf.noarch 0:9-3.el6 will be installed
--> Finished Dependency Resolution

Dependencies Resolved

=====
Package Arch Version Repository
=====
Installing:
pydf noarch 9-3.el6 epel

Transaction Summary
=====
Install 1 Package(s)

Total download size: 14 k
Installed size: 25 k
Is this ok [y/N]: y
Downloading Packages:
pydf-9-3.el6.noarch.rpm | 14 kB
Running rpm_check_debug
Running Transaction Test
Transaction Test Succeeded
Running Transaction
  Installing : pydf-9-3.el6.noarch 1/1
  Verifying : pydf-9-3.el6.noarch 1/1

Installed:
pydf.noarch 0:9-3.el6
```


On Debian/Ubuntu/Linux Mint

On **Debian** based distributions, the pydf tool is available from package manager system, you can install it using `apt-get` command as shown below.

```
$ sudo apt-get install pydf
```

Sample Output

```
[sudo] password for tecmint:

Reading package lists... Done

Building dependency tree

Reading state information... Done

The following package was automatically installed and is
no longer required:

    java-wrappers

Use 'apt-get autoremove' to remove it.

The following NEW packages will be installed:

    pydf

0 upgraded, 1 newly installed, 0 to remove and 10 not
upgraded.

Need to get 12.1 kB of archives.

After this operation, 70.7 kB of additional disk space
will be used.
```



```

Get:1 http://in.archive.ubuntu.com/ubuntu/
raring/universe pydf all 10 [12.1 kB]

Fetched 12.1 kB in 1s (6,097 B/s)

Selecting previously unselected package pydf.

(Reading database ... 175568 files and directories
currently installed.)

Unpacking pydf (from .../apt/archives/pydf_10_all.deb)
...

Processing triggers for man-db ...

Setting up pydf (10) ...

```

How to Use pydf Command

1. If you run only `-pydfll` command without specifying argument, it will display just information of disk space usage along with all mounted file systems.

```
# pydf
```

Filesystem	Size	Used	Avail	Use%		Mounted on
/dev/sda2	49G	3244M	44G	6.4	[###.....]	/
/dev/sda1	194M	43M	140M	22.4	[#####.....]	/boot
/dev/sdb1	492G	345G	122G	70.2	[#####.....]	/data
/dev/sda3	39G	4043M	33G	10.0	[###.....]	/home
/dev/sda6	55G	33G	19G	60.0	[#####.....]	/var
/dev/sdc1	492G	262G	205G	53.3	[#####.....]	/videos

2. To see the output in human readable format, use the option `--hll`, which will print sizes (e.g., 10K, 10M, 10G).

```
# pydf -h
```

Filesystem	Size	Used	Avail	Use%		Mounted on
/dev/sda2	49G	3244M	44G	6.4	[###.....]	/
/dev/sda1	194M	43M	140M	22.4	[#####.....]	/boot
/dev/sdb1	492G	345G	122G	70.2	[#####.....]	/data
/dev/sda3	39G	4043M	33G	10.0	[###.....]	/home
/dev/sda6	55G	33G	19G	60.0	[#####.....]	/var
/dev/sdc1	492G	262G	205G	53.3	[#####.....]	/videos

3. To print sizes in kilobytes (e.g., **1024** block size) use option `--kll` as shown below.

```
# pydf -k
```

Filesystem	Size	Used	Avail	Use%		Mount
/dev/sda2	51606140	3321932	45662768	6.4	[##.....]	/
/dev/sda1	198337	44371	143726	22.4	[#####.....]	/boot
/dev/sdb1	516054864	362172932	127667872	70.2	[#####.....]	/data
/dev/sda3	41284928	4140032	35047744	10.0	[###.....]	/home
/dev/sda6	57593092	34575244	20092232	60.0	[#####.....]	/var
/dev/sdc1	516054864	274980776	214860028	53.3	[#####.....]	/vid

4. To print summary output in megabytes (e.g., **1048576** block size) use option **-m** as shown below.

```
pydf -m
```

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/sda2	50397	3244	44593	6.4	[##.....] /
/dev/sda1	194	43	140	22.4	[#####.....] /boot
/dev/sdb1	503960	353685	124675	70.2	[#####.....] /data
/dev/sda3	40317	4043	34226	10.0	[####.....] /home
/dev/sda6	56243	33768	19618	60.0	[#####.....] /var
/dev/sdc1	503960	268536	209824	53.3	[#####.....] /videos

To print sizes in gigabytes (e.g., **1073741824**), specify option **-g** as shown below.

```
# pydf -g
```

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/sda2	50397	3244	44593	6.4	[##.....] /
/dev/sda1	194	43	140	22.4	[#####.....] /boot
/dev/sdb1	503960	353685	124675	70.2	[#####.....] /data
/dev/sda3	40317	4043	34226	10.0	[####.....] /home
/dev/sda6	56243	33770	19616	60.0	[#####.....] /var
/dev/sdc1	503960	268536	209824	53.3	[#####.....] /videos

To see the information about **inodes** instead of blocks, use the argument “-i” as given below.

```
# pydf -i
```

```
Filesystem Nodes  Used Avail Use% Mounted on
/dev/sda2 3200k   78k 3122k  2.5 [#.....] /
/dev/sda1   50k    40   50k  0.1 [.....] /boot
/dev/sdb1   31M   411k   31M  1.3 [#.....] /data
/dev/sda3 2560k  3031 2557k  0.1 [.....] /home
/dev/sda6 3576k  873k 2703k 24.4 [#####.....] /var
/dev/sdc1   31M 2033k   29M  6.4 [###.....] /videos
```

If you want to disable coloured output, simply use the option “-bw”.

```
# pydf --bw
```

```
Filesystem Size  Used Avail Use% Mounted on
/dev/sda2  49G 3244M   44G  6.4 [###.....] /
/dev/sda1  194M   43M  140M 22.4 [#####.....] /boot
/dev/sdb1  492G  345G  122G 70.2 [#####.....] /data
/dev/sda3   39G 4043M   33G 10.0 [###.....] /home
/dev/sda6   55G   33G   19G 60.1 [#####.....] /var
/dev/sdc1  492G  262G  205G 53.3 [#####.....] /videos
```

C. Linux commands for disk management: parted

Parted is a command line tool for managing disk partitions on Linux.

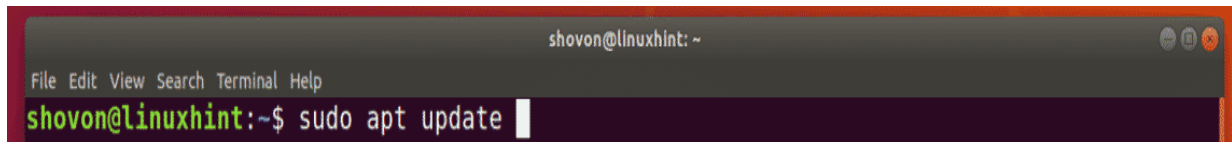
Parted can be used to work with both MSDOS and GPT partition tables. Parted can be used to do many low level partitioning tasks easily.

GNU parted (**PARTition Editor**) runs at the command line, or with a graphical interface. Parted allows you to add, shrink, extend, and remove partitions from storage disks on your system.

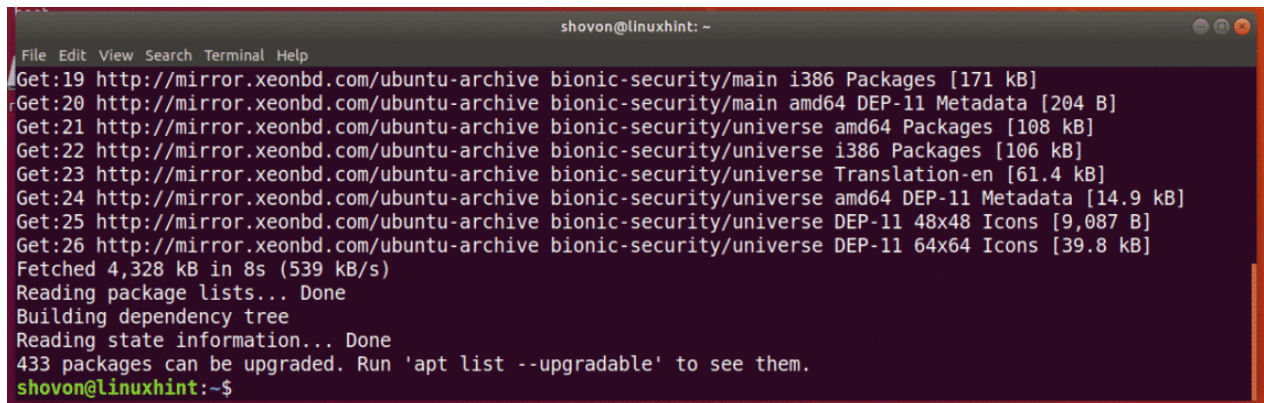
Installing parted:

If you're using Ubuntu or any Debian based Linux distributions, then you can easily install parted as it is available in the official package repository. First, update the APT package repository cache with the following command:

```
$ sudo apt update
```

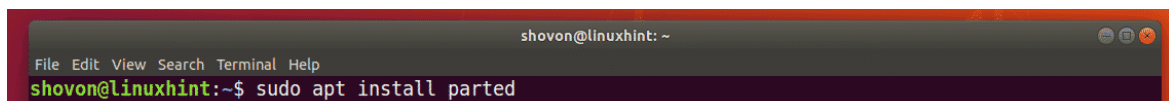
A terminal window titled 'shovon@linuxhint: ~' with a menu bar (File, Edit, View, Search, Terminal, Help). The prompt is 'shovon@linuxhint:~\$' and the command 'sudo apt update' is being typed, with a cursor at the end of the line.

The APT package repository cache is updated.

A terminal window titled 'shovon@linuxhint: ~' showing the output of 'sudo apt update'. The output lists updates for various packages from the bionic-security repository, including metadata and icons. It shows that 433 packages can be upgraded. The prompt returns to 'shovon@linuxhint:~\$'.

Now, run the following command to install parted:

```
$ sudo apt install parted
```

A terminal window titled 'shovon@linuxhint: ~' with a menu bar (File, Edit, View, Search, Terminal, Help). The prompt is 'shovon@linuxhint:~\$' and the command 'sudo apt install parted' is being typed, with a cursor at the end of the line.

Now, press **y** and then press **<Enter>** to continue.

```
shovon@linuxhint: ~  
File Edit View Search Terminal Help  
Building dependency tree  
Reading state information... Done  
The following additional packages will be installed:  
  libparted-fs-resize0 libparted2  
Suggested packages:  
  libparted-dev libparted-i18n parted-doc  
The following packages will be upgraded:  
  libparted-fs-resize0 libparted2 parted  
3 upgraded, 0 newly installed, 0 to remove and 430 not upgraded.  
Need to get 0 B/212 kB of archives.  
After this operation, 0 B of additional disk space will be used.  
Do you want to continue? [Y/n]
```

Parted should be installed.

```
shovon@linuxhint: ~  
File Edit View Search Terminal Help  
Preparing to unpack .../libparted-fs-resize0_3.2-20ubuntu0.1_amd64.deb ...  
Unpacking libparted-fs-resize0:amd64 (3.2-20ubuntu0.1) over (3.2-20) ...  
Preparing to unpack .../libparted2_3.2-20ubuntu0.1_amd64.deb ...  
Unpacking libparted2:amd64 (3.2-20ubuntu0.1) over (3.2-20) ...  
Setting up libparted2:amd64 (3.2-20ubuntu0.1) ...  
Setting up libparted-fs-resize0:amd64 (3.2-20ubuntu0.1) ...  
Setting up parted (3.2-20ubuntu0.1) ...  
Processing triggers for libc-bin (2.27-3ubuntu1) ...  
Processing triggers for man-db (2.8.3-2) ...  
shovon@linuxhint:~$
```

On CentOS/RHEL 7, you can install parted with the following command:

```
$ sudo yum install parted -y
```

Finding Storage Device Identifiers:

Before you can start working with parted, you have to know which storage device you need to partition.

You can run the following command to list all the attached storage devices on your computer:

```
$ sudo lshw -class disk -short
```

As you can see, WE have 2 storage devices on my computer, **/dev/sda** and **/dev/sdb**. Here, **/dev/sdb** is my 32GB USB thumb drive. This is the one WE want to partition.

```
shovon@linuxhint: ~  
File Edit View Search Terminal Help  
shovon@linuxhint:~$ sudo lshw -class disk -short  
H/W path          Device          Class        Description  
=====
```

H/W path	Device	Class	Description
/0/100/10/0.0.0	/dev/sda	disk	21GB VMware Virtual S
/0/100/15/0/0/2/0.0.0	/dev/sdb	disk	31GB Transcend 32GB
/0/100/15/0/0/2/0.0.0/0	/dev/sdb	disk	31GB

```
shovon@linuxhint:~$
```

Note: In the original image, green boxes highlight /dev/sdb and 31GB Transcend 32GB, and green arrows point to them.

Opening Storage Device with parted:

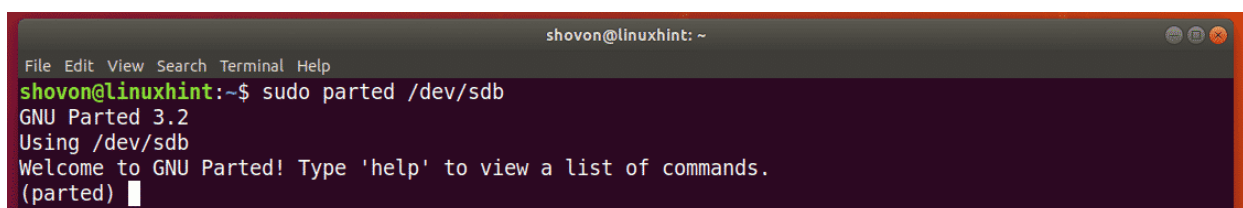
Now that you know which storage device you want to partition, you can open parted as follows:

```
$ sudo parted /dev/sdb
```

NOTE: Make sure you change **/dev/sdb** to the storage device that you want to partition.

A terminal window titled 'shovon@linuxhint: ~' with a menu bar (File, Edit, View, Search, Terminal, Help). The command 'shovon@linuxhint:~\$ sudo parted /dev/sdb' is entered and executed. The prompt changes to '(parted) '.

Parted should be opened. Now, you can run many of the parted commands to partition your desired storage device any way you want.

A terminal window titled 'shovon@linuxhint: ~' with a menu bar. The command 'shovon@linuxhint:~\$ sudo parted /dev/sdb' is entered. The output is: 'GNU Parted 3.2', 'Using /dev/sdb', 'Welcome to GNU Parted! Type \'help\' to view a list of commands.', and the prompt '(parted) '.

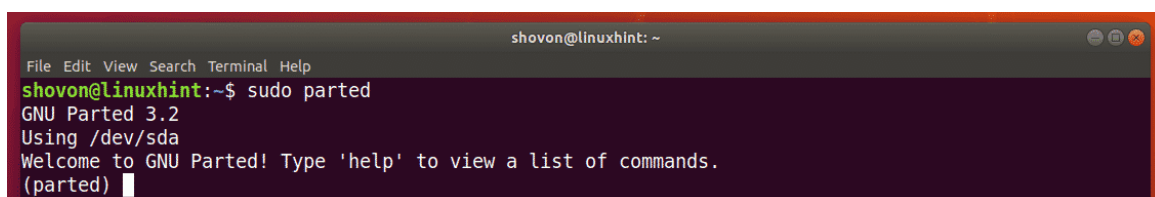
Switching to Different Storage Device:

You can also start parted without specifying which storage device to open beforehand as follows:

```
$ sudo parted
```

A terminal window titled 'shovon@linuxhint: ~' with a menu bar. The command 'shovon@linuxhint:~\$ sudo parted' is entered and executed. The prompt changes to '(parted) '.

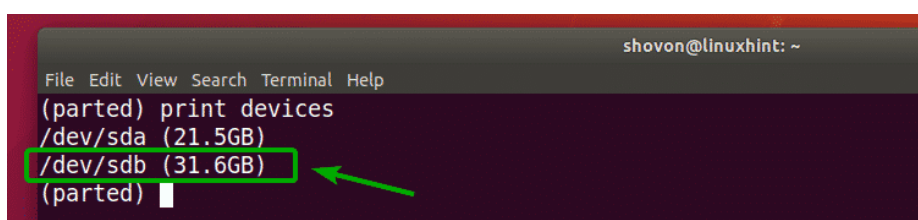
As you can see, parted is started. By default, **/dev/sda**, the first/primary storage device is selected.

A terminal window titled 'shovon@linuxhint: ~' with a menu bar. The command 'shovon@linuxhint:~\$ sudo parted' is entered. The output is: 'GNU Parted 3.2', 'Using /dev/sda', 'Welcome to GNU Parted! Type \'help\' to view a list of commands.', and the prompt '(parted) '.

You can list all the storage devices on your computer with the following parted command:

```
(parted) print devices
```

As you can see, the storage devices on my computer **/dev/sda** and **/dev/sdb** are listed along with their physical size.

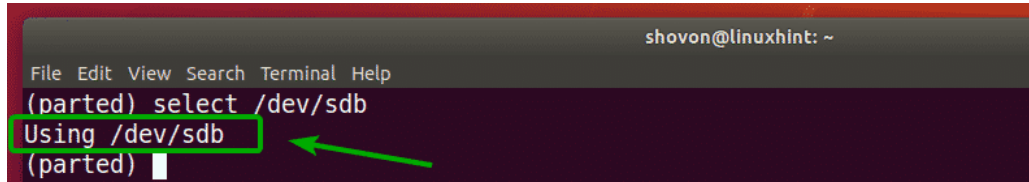
A terminal window titled 'shovon@linuxhint: ~' with a menu bar. The command '(parted) print devices' is entered. The output is: '/dev/sda (21.5GB)', '/dev/sdb (31.6GB)', and the prompt '(parted) '. A green box highlights the output for /dev/sdb, and a green arrow points to it.

Now, you can use the **select** parted command to select the storage device (let's say **/dev/sdb**) that you want to partition as follows:

(parted) select /dev/sdb

A terminal window titled 'shovon@linuxhint: ~' with a menu bar (File, Edit, View, Search, Terminal, Help). The command '(parted) select /dev/sdb' is entered and highlighted in white text on a dark background.

As you can see, **/dev/sdb** is selected.

A terminal window titled 'shovon@linuxhint: ~' with a menu bar. The command '(parted) select /dev/sdb' is entered. Below it, the message 'Using /dev/sdb' is displayed in white text, highlighted with a green box. A green arrow points to this message. The prompt '(parted) ' is visible at the end of the line.

Creating a New Partition Table:

You can create GPT and MSDOS partition table with parted.

To create a GPT partition table, run the following parted command:

(parted) mklabel gpt

A terminal window titled 'shovon@linuxhint: ~' with a menu bar. The command '(parted) mklabel gpt' is entered and highlighted in white text on a dark background.

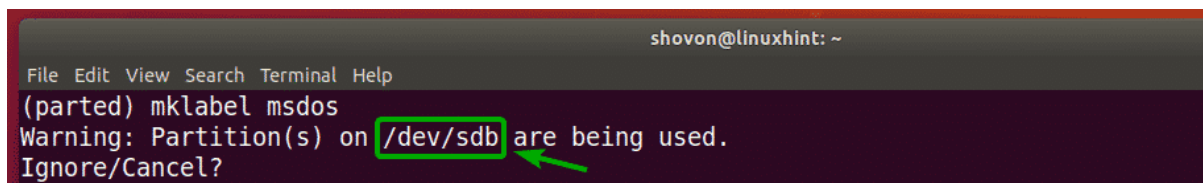
To create a MSDOS partition table, run the following parted command:

(parted) mklabel msdos

A terminal window titled 'shovon@linuxhint: ~' with a menu bar. The command '(parted) mklabel msdos' is entered and highlighted in white text on a dark background.

I will go for MSDOS partition table as I am partitioning a USB thumb drive. The procedure for the GPT partition creation is the same.

Now, type in **Ignore** and press **<Enter>**.

A terminal window titled 'shovon@linuxhint: ~' with a menu bar. The command '(parted) mklabel msdos' is entered. Below it, a warning message is displayed: 'Warning: Partition(s) on /dev/sdb are being used. Ignore/Cancel?'. The text '/dev/sdb' is highlighted with a green box, and a green arrow points to it. The prompt '(parted) ' is visible at the end of the line.

When you create a new partition table, all the existing partitions will be erased. If you're okay with it, type in **Yes** and then press **<Enter>**.


```
shovon@linuxhint: ~  
File Edit View Search Terminal Help  
(parted) mklabel msdos  
Warning: Partition(s) on /dev/sdb are being used.  
Ignore/Cancel? Ignore  
Warning: The existing disk label on /dev/sdb will be destroyed and all data on this disk will be  
lost. Do you want to continue?  
Yes/No? █
```

For some reason, the changes can't be applied immediately. But it's alright. Type in Ignore and press <Enter>.

```
shovon@linuxhint: ~  
File Edit View Search Terminal Help  
(parted) mklabel msdos  
Warning: Partition(s) on /dev/sdb are being used.  
Ignore/Cancel? Ignore  
Warning: The existing disk label on /dev/sdb will be destroyed and all data on this disk will be  
lost. Do you want to continue?  
Yes/No? Yes  
Error: Partition(s) 1, 2 on /dev/sdb have been written, but we have been unable to inform the  
kernel of the change, probably because it/they are in use. As a result, the old partition(s) will  
remain in use. You should reboot now before making further changes.  
Ignore/Cancel? █
```

A new partition table should be created.

```
shovon@linuxhint: ~  
File Edit View Search Terminal Help  
(parted) mklabel msdos  
Warning: Partition(s) on /dev/sdb are being used.  
Ignore/Cancel? Ignore  
Warning: The existing disk label on /dev/sdb will be destroyed and all data on this disk will be  
lost. Do you want to continue?  
Yes/No? Yes  
Error: Partition(s) 1, 2 on /dev/sdb have been written, but we have been unable to inform the  
kernel of the change, probably because it/they are in use. As a result, the old partition(s) will  
remain in use. You should reboot now before making further changes.  
Ignore/Cancel? Ignore  
(parted) █
```

Creating New Partitions:

You can create a new partition with the following parted command:

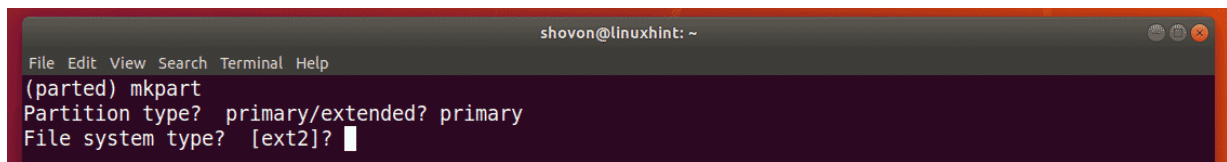
(parted) mkpart

```
shovon@linuxhint: ~  
File Edit View Search Terminal Help  
(parted) mkpart
```

Now, type in either **primary** or **extended** depending on whether you want to create a primary or extended partition. Once you're done, press <Enter>.

```
shovon@linuxhint: ~  
File Edit View Search Terminal Help  
(parted) mkpart  
Partition type? primary/extended? █
```

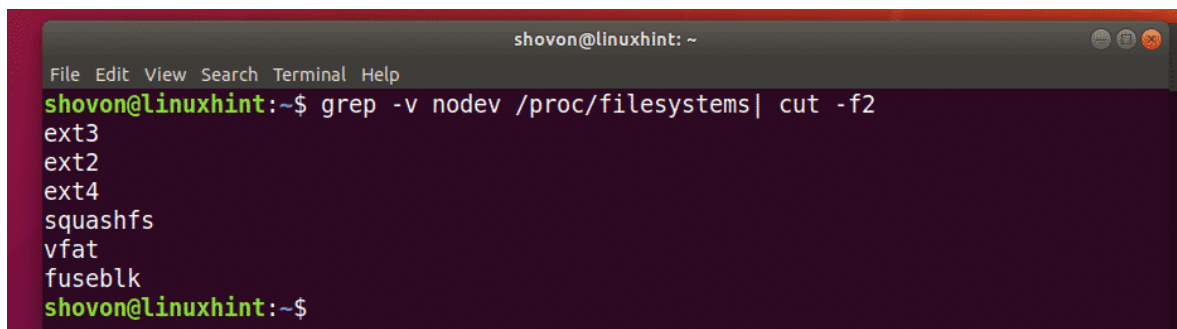
Now, type in a filesystem type that you want to use for the partition. We will go for **ext4**.



```
shovon@linuxhint: ~  
File Edit View Search Terminal Help  
(parted) mkpart  
Partition type? primary/extended? primary  
File system type? [ext2]?
```

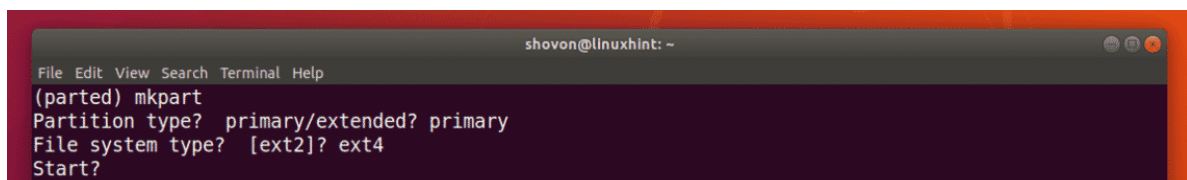
NOTE: You can find out what keywords you can use here with the following command:

```
$ grep -v nodev /proc/filesystems| cut -f2
```



```
shovon@linuxhint: ~  
File Edit View Search Terminal Help  
shovon@linuxhint:~$ grep -v nodev /proc/filesystems| cut -f2  
ext3  
ext2  
ext4  
squashfs  
vfat  
fuseblk  
shovon@linuxhint:~$
```

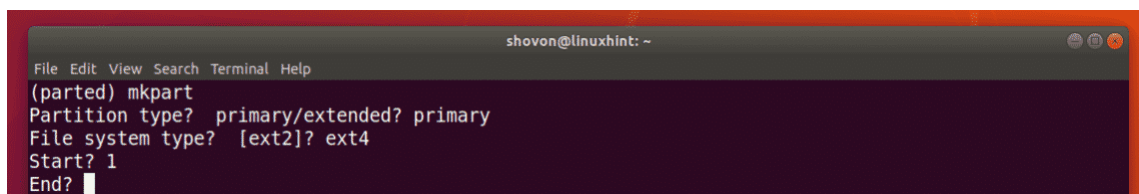
Now, type in the location in megabyte (MB) where the partition starts. If it's the first partition, then 1 (MB) is an acceptable value. Once you're done, press **<Enter>**.



```
shovon@linuxhint: ~  
File Edit View Search Terminal Help  
(parted) mkpart  
Partition type? primary/extended? primary  
File system type? [ext2]? ext4  
Start?
```

Now, type in the location in megabyte (MB) where the partition ends. The size of the partition will be the difference between the End and Start location. For example, let's say, you want to create a 1GB/1024MB partition. So, the end will be 1024. Once you're done, press **<Enter>**.

NOTE: You can't put 1025 here due to alignment problems. Parted don't align partitions automatically.



```
shovon@linuxhint: ~  
File Edit View Search Terminal Help  
(parted) mkpart  
Partition type? primary/extended? primary  
File system type? [ext2]? ext4  
Start? 1  
End?
```

The partition will be created.

```
shovon@linuxhint: ~  
File Edit View Search Terminal Help  
(parted) mkpart  
Partition type? primary/extended? primary  
File system type? [ext2]? ext4  
Start? 1  
End? 1024  
(parted) █
```

You can list all the partitions of your selected storage devices as follows:

(parted) print

```
shovon@linuxhint: ~  
File Edit View Search Terminal Help  
(parted) print
```

As you can see, the newly created partition is listed.

```
shovon@linuxhint: ~  
File Edit View Search Terminal Help  
(parted) print  
Model: JetFlash Transcend 32GB (scsi)  
Disk /dev/sdb: 31.6GB  
Sector size (logical/physical): 512B/512B  
Partition Table: msdos  
Disk Flags:  


| Number | Start  | End    | Size   | Type    | File system | Flags |
|--------|--------|--------|--------|---------|-------------|-------|
| 1      | 1049kB | 1024MB | 1023MB | primary | ext4        | lba   |

  
(parted)
```

NOTE: When you create multiple partitions with parted, you have to start the new partition from at least End+1 of the last partition. For example, the partition we created earlier ended in 1024MB. So, the next partition will start from 1025MB or more.

We created another partition to demonstrate how to remove partitions using parted in the next section.

```
shovon@linuxhint: ~  
File Edit View Search Terminal Help  
(parted) mkpart  
Partition type? primary/extended? primary  
File system type? [ext2]? ext4  
Start? 1030  
End? 10000  
(parted) print  
Model: JetFlash Transcend 32GB (scsi)  
Disk /dev/sdb: 31.6GB  
Sector size (logical/physical): 512B/512B  
Partition Table: msdos  
Disk Flags:  
  
Number  Start   End     Size    Type    File system  Flags  
1       1049kB  1024MB  1023MB  primary ext4          lba  
2       1030MB  10.0GB  8971MB  primary ext4          lba  
  
(parted) █
```

Removing Partitions:

First, list all the partitions of your selected storage device as follows:

(parted) print

Let's say, you want to delete the partition number 2 as marked in the screenshot below.

```
shovon@linuxhint: ~  
File Edit View Search Terminal Help  
(parted) mkpart  
Partition type? primary/extended? primary  
File system type? [ext2]? ext4  
Start? 1030  
End? 10000  
(parted) print  
Model: JetFlash Transcend 32GB (scsi)  
Disk /dev/sdb: 31.6GB  
Sector size (logical/physical): 512B/512B  
Partition Table: msdos  
Disk Flags:  
  
Number  Start   End     Size    Type    File system  Flags  
1       1049kB  1024MB  1023MB  primary ext4          lba  
2       1030MB  10.0GB  8971MB  primary ext4          lba  
  
(parted) █
```

To do that, run the following parted command:

(parted) rm 2

```
shovon@linuxhint: ~  
File Edit View Search Terminal Help  
(parted) rm 2  
(parted) █
```

As you can see, partition number 2 no longer exists.

```
shovon@linuxhint: ~  
File Edit View Search Terminal Help  
(parted) rm 2  
(parted) print  
Model: JetFlash Transcend 32GB (scsi)  
Disk /dev/sdb: 31.6GB  
Sector size (logical/physical): 512B/512B  
Partition Table: msdos  
Disk Flags:  
  
Number  Start   End     Size    Type    File system  Flags  
1       1049kB  1024MB  1023MB  primary ext4         lba  
  
(parted) █
```

Changing the Unit:

When you create a new partition, you have to specify the Start and End section of your new partition. The default unit is MB. You can change it very easily in parted.

The supported units and keywords are:

Unit	keyword
Sectors	s
Bytes	B
Cylinders	cyl
cylinders, heads, sectors	chs
Kilobytes	KB
Mebibytes	MiB
Megabytes	MB
Gibibytes	GiB
Gigabytes	GB
Percentage	%

NOTE: For more information on this, check the man page of parted with the following command:

```
$ man parted
```

You can use the unit command to change the default unit.

For example, let's say you want to change the default unit MB to sectors, then run the following command:

```
(parted) unit s
```

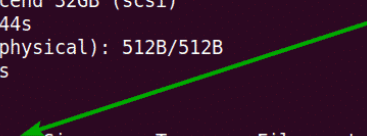

As you can see, the display unit has changed as well.

```
shovon@linuxhint: ~  
File Edit View Search Terminal Help  
(parted)  
(parted) print  
Model: JetFlash Transcend 32GB (scsi)  
Disk /dev/sdb: 31.6GB  
Sector size (logical/physical): 512B/512B  
Partition Table: msdos  
Disk Flags:  


| Number | Start  | End    | Size   | Type    | File system | Flags |
|--------|--------|--------|--------|---------|-------------|-------|
| 1      | 1049kB | 1024MB | 1023MB | primary | ext4        | lba   |

  
(parted) unit s  
(parted) print  
Model: JetFlash Transcend 32GB (scsi)  
Disk /dev/sdb: 61702144s  
Sector size (logical/physical): 512B/512B  
Partition Table: msdos  
Disk Flags:  


| Number | Start | End      | Size     | Type    | File system | Flags |
|--------|-------|----------|----------|---------|-------------|-------|
| 1      | 2048s | 2000895s | 1998848s | primary | ext4        | lba   |

  
(parted) 
```

Now, you can also create partitions with the newly set unit.

```
shovon@linuxhint: ~  
File Edit View Search Terminal Help  
(parted) mkpart  
Partition type? primary/extended? primary  
File system type? [ext2]? ext4  
Start? 2048  
End? 2000895  
(parted) print  
Model: JetFlash Transcend 32GB (scsi)  
Disk /dev/sdb: 61702144s  
Sector size (logical/physical): 512B/512B  
Partition Table: msdos  
Disk Flags:  


| Number | Start | End      | Size     | Type    | File system | Flags |
|--------|-------|----------|----------|---------|-------------|-------|
| 1      | 2048s | 2000895s | 1998848s | primary | ext4        | lba   |

  
(parted)
```

D. Linux commands for disk management: lsblk,

Lsblk is used to display details about block devices and these block devices(Except ram disk) are basically those files that represent devices connected to the pc. It queries /sys virtual file system and udev db to obtain information that it displays. And it basically displays output in a tree-like structure. This command comes pre-installed with the util-linux package.

Installing lsblk Command

Many Linux distributions do not have lsblk command pre-installed. To install it use the following commands as per your Linux distribution.

In case of Debian/Ubuntu

```
$sudo apt-get install util-linux
```

In case of CentOS/RedHat

```
$sudo yum install util-linux-ng
```

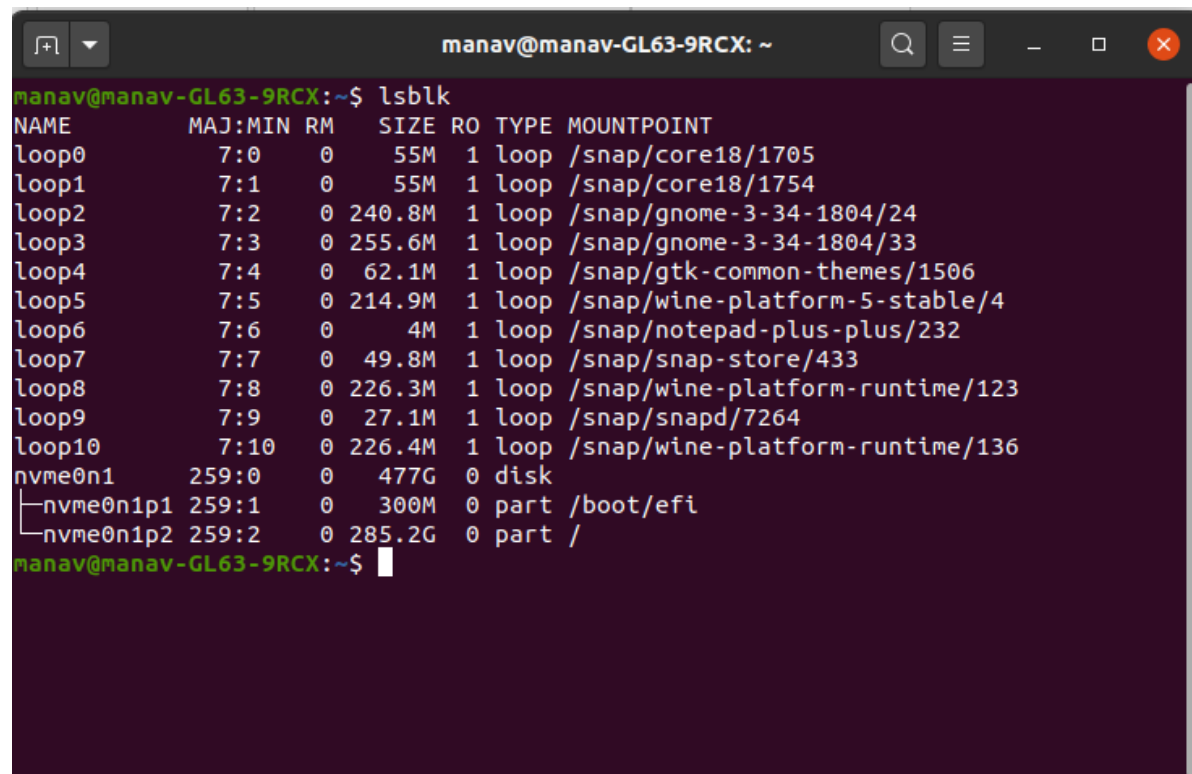
In case of Fedora OS

```
$sudo yum install util-linux-ng
```

Working with lsblk command

1. To display block devices.

```
$lsblk
```

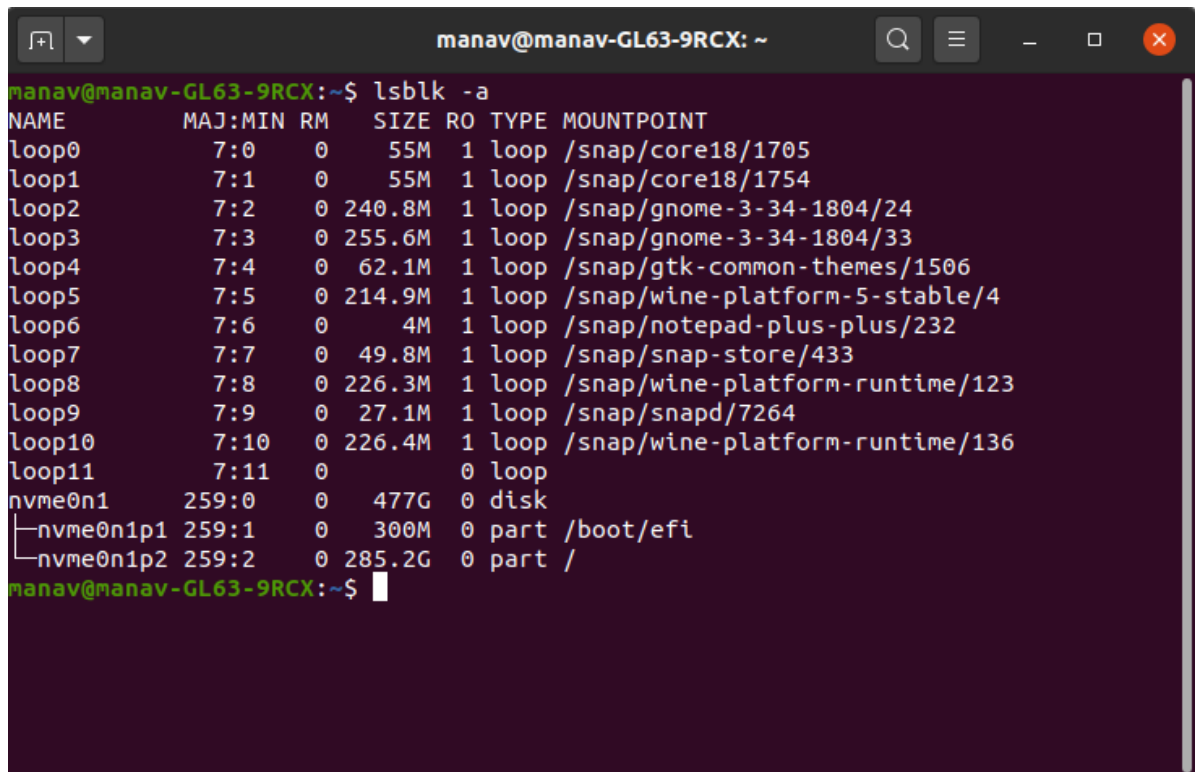


```
manav@manav-GL63-9RCX:~$ lsblk
NAME                MAJ:MIN RM   SIZE RO TYPE MOUNTPOINT
loop0                7:0    0    55M  1 loop /snap/core18/1705
loop1                7:1    0    55M  1 loop /snap/core18/1754
loop2                7:2    0 240.8M  1 loop /snap/gnome-3-34-1804/24
loop3                7:3    0 255.6M  1 loop /snap/gnome-3-34-1804/33
loop4                7:4    0   62.1M  1 loop /snap/gtk-common-themes/1506
loop5                7:5    0 214.9M  1 loop /snap/wine-platform-5-stable/4
loop6                7:6    0     4M  1 loop /snap/notepad-plus-plus/232
loop7                7:7    0   49.8M  1 loop /snap/snap-store/433
loop8                7:8    0  226.3M  1 loop /snap/wine-platform-runtime/123
loop9                7:9    0   27.1M  1 loop /snap/snapd/7264
loop10              7:10    0  226.4M  1 loop /snap/wine-platform-runtime/136
nvme0n1             259:0    0   477G  0 disk
├─nvme0n1p1         259:1    0   300M  0 part /boot/efi
└─nvme0n1p2         259:2    0  285.2G  0 part /
manav@manav-GL63-9RCX:~$
```

It displays the list of block devices on your system.

2. To display empty block devices as well.

#lsblk -a

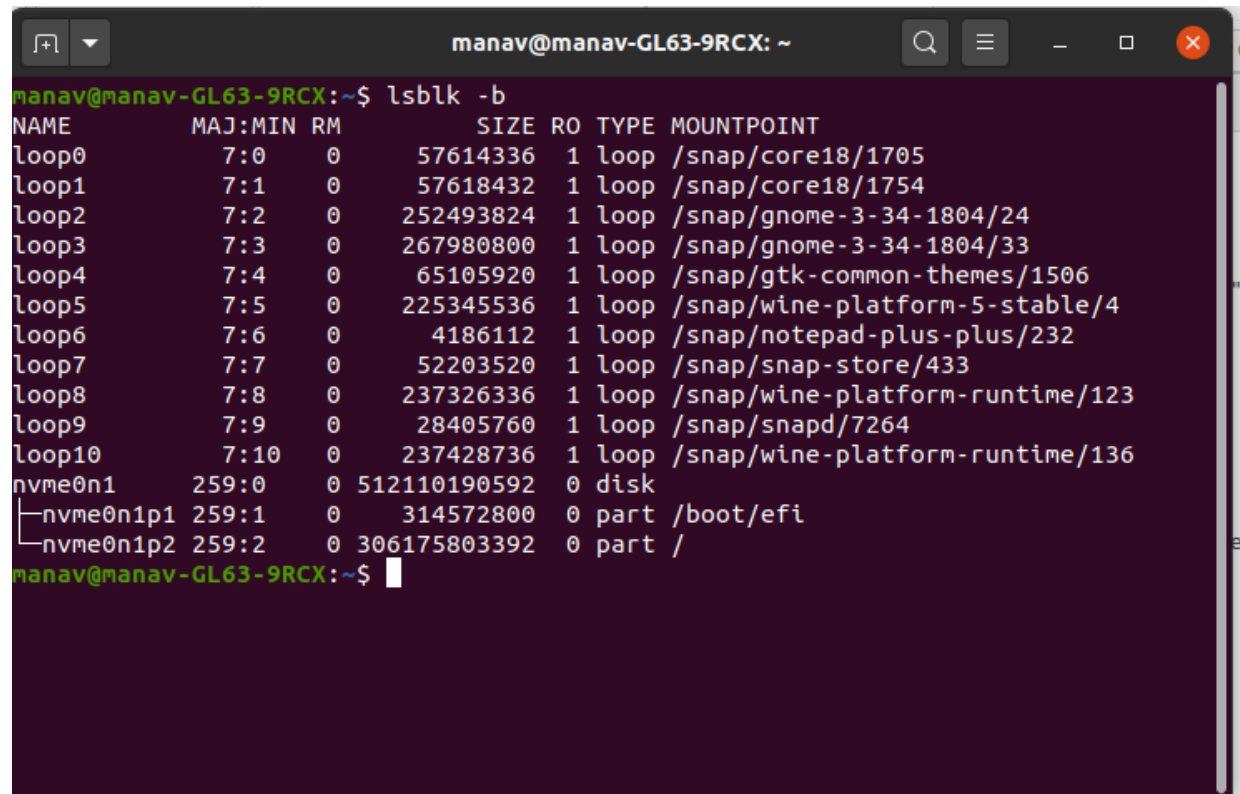
A terminal window titled 'manav@manav-GL63-9RCX: ~' with standard window controls. The command 'lsblk -a' has been executed, displaying a table of block devices. The table includes columns for NAME, MAJ:MIN, RM, SIZE, RO, TYPE, and MOUNTPOINT. It lists various loop devices (loop0 through loop11) and NVMe storage (nvme0n1 and its partitions).

```
manav@manav-GL63-9RCX:~$ lsblk -a
NAME                MAJ:MIN RM   SIZE RO TYPE MOUNTPOINT
loop0                7:0      0    55M  1 loop /snap/core18/1705
loop1                7:1      0    55M  1 loop /snap/core18/1754
loop2                7:2      0 240.8M  1 loop /snap/gnome-3-34-1804/24
loop3                7:3      0 255.6M  1 loop /snap/gnome-3-34-1804/33
loop4                7:4      0   62.1M  1 loop /snap/gtk-common-themes/1506
loop5                7:5      0 214.9M  1 loop /snap/wine-platform-5-stable/4
loop6                7:6      0     4M  1 loop /snap/notepad-plus-plus/232
loop7                7:7      0   49.8M  1 loop /snap/snap-store/433
loop8                7:8      0 226.3M  1 loop /snap/wine-platform-runtime/123
loop9                7:9      0   27.1M  1 loop /snap/snapd/7264
loop10               7:10     0 226.4M  1 loop /snap/wine-platform-runtime/136
loop11               7:11     0      0 loop
nvme0n1              259:0     0   477G  0 disk
├─nvme0n1p1          259:1     0   300M  0 part /boot/efi
└─nvme0n1p2          259:2     0 285.2G  0 part /
```

This will display all block devices along with empty ones.

3. To print size information in bytes.

`$lsblk -b`

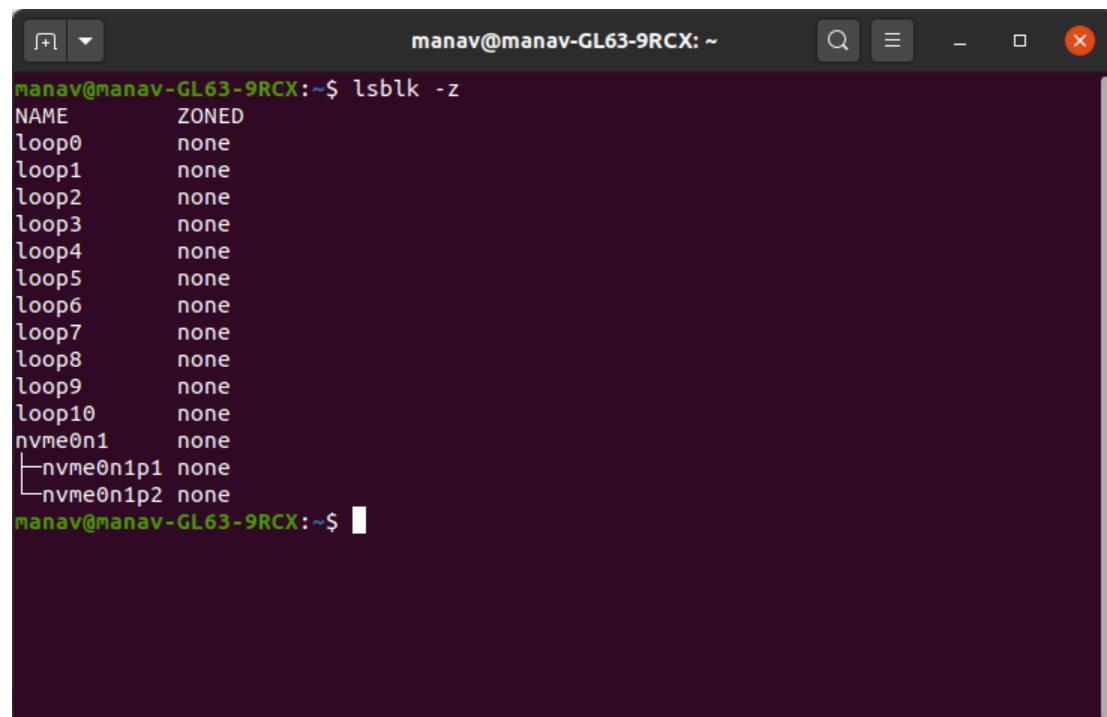


```
manav@manav-GL63-9RCX: ~$ lsblk -b
NAME                MAJ:MIN RM   SIZE RO TYPE MOUNTPOINT
loop0                 7:0    0  57614336 1 loop /snap/core18/1705
loop1                 7:1    0  57618432 1 loop /snap/core18/1754
loop2                 7:2    0 252493824 1 loop /snap/gnome-3-34-1804/24
loop3                 7:3    0 267980800 1 loop /snap/gnome-3-34-1804/33
loop4                 7:4    0  65105920 1 loop /snap/gtk-common-themes/1506
loop5                 7:5    0 225345536 1 loop /snap/wine-platform-5-stable/4
loop6                 7:6    0  4186112  1 loop /snap/notepad-plus-plus/232
loop7                 7:7    0  52203520 1 loop /snap/snap-store/433
loop8                 7:8    0 237326336 1 loop /snap/wine-platform-runtime/123
loop9                 7:9    0  28405760 1 loop /snap/snapd/7264
loop10                7:10   0  237428736 1 loop /snap/wine-platform-runtime/136
nvme0n1              259:0    0 512110190592 0 disk
├─nvme0n1p1          259:1    0  314572800 0 part /boot/efi
└─nvme0n1p2          259:2    0 306175803392 0 part /
```

It displays all the block devices along with their sizes in bytes.

4. To print zone model for devices.

`$lsblk -z`



```
manav@manav-GL63-9RCX: ~$ lsblk -z
NAME                ZONED
loop0                none
loop1                none
loop2                none
loop3                none
loop4                none
loop5                none
loop6                none
loop7                none
loop8                none
loop9                none
loop10               none
nvme0n1              none
├─nvme0n1p1          none
└─nvme0n1p2          none
```

5. To skip slave entries.

`$lsblk -d`

```
manav@manav-GL63-9RCX:~$ lsblk -d
NAME        MAJ:MIN RM   SIZE RO TYPE MOUNTPOINT
loop0        7:0      0    55M  1 loop /snap/core18/1705
loop1        7:1      0    55M  1 loop /snap/core18/1754
loop2        7:2      0 240.8M  1 loop /snap/gnome-3-34-1804/24
loop3        7:3      0 255.6M  1 loop /snap/gnome-3-34-1804/33
loop4        7:4      0   62.1M  1 loop /snap/gtk-common-themes/1506
loop5        7:5      0 214.9M  1 loop /snap/wine-platform-5-stable/4
loop6        7:6      0     4M  1 loop /snap/notepad-plus-plus/232
loop7        7:7      0   49.8M  1 loop /snap/snap-store/433
loop8        7:8      0 226.3M  1 loop /snap/wine-platform-runtime/123
loop9        7:9      0   27.1M  1 loop /snap/snapd/7264
loop10       7:10     0 226.4M  1 loop /snap/wine-platform-runtime/136
nvme0n1     259:0    0   477G  0 disk
```

This will skip all the slave entries.

6. To use ASCII characters for tree formatting.

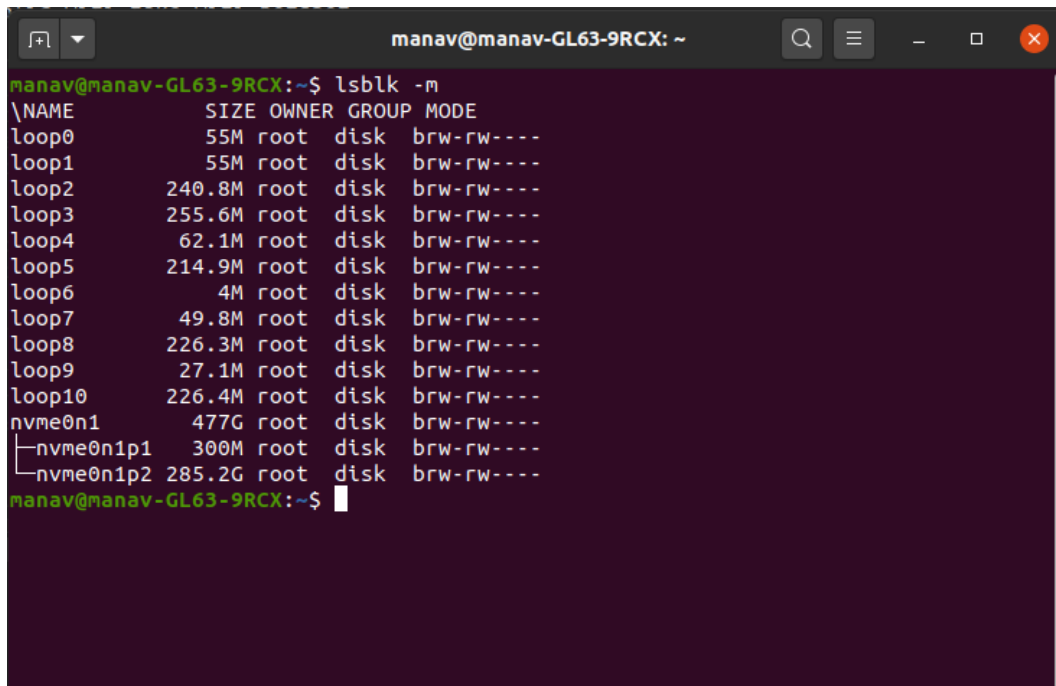
`$lsblk -i`

```
manav@manav-GL63-9RCX:~$ lsblk -i
NAME        MAJ:MIN RM   SIZE RO TYPE MOUNTPOINT
loop0        7:0      0    55M  1 loop /snap/core18/1705
loop1        7:1      0    55M  1 loop /snap/core18/1754
loop2        7:2      0 240.8M  1 loop /snap/gnome-3-34-1804/24
loop3        7:3      0 255.6M  1 loop /snap/gnome-3-34-1804/33
loop4        7:4      0   62.1M  1 loop /snap/gtk-common-themes/1506
loop5        7:5      0 214.9M  1 loop /snap/wine-platform-5-stable/4
loop6        7:6      0     4M  1 loop /snap/notepad-plus-plus/232
loop7        7:7      0   49.8M  1 loop /snap/snap-store/433
loop8        7:8      0 226.3M  1 loop /snap/wine-platform-runtime/123
loop9        7:9      0   27.1M  1 loop /snap/snapd/7264
loop10       7:10     0 226.4M  1 loop /snap/wine-platform-runtime/136
nvme0n1     259:0    0   477G  0 disk
|-nvme0n1p1 259:1    0   300M  0 part /boot/efi
`-nvme0n1p2 259:2    0 285.2G  0 part /
```

This will display the tree formatting with ASCII character values.

7. To print information about device owner, group, and mode of block devices.

\$lsblk -m

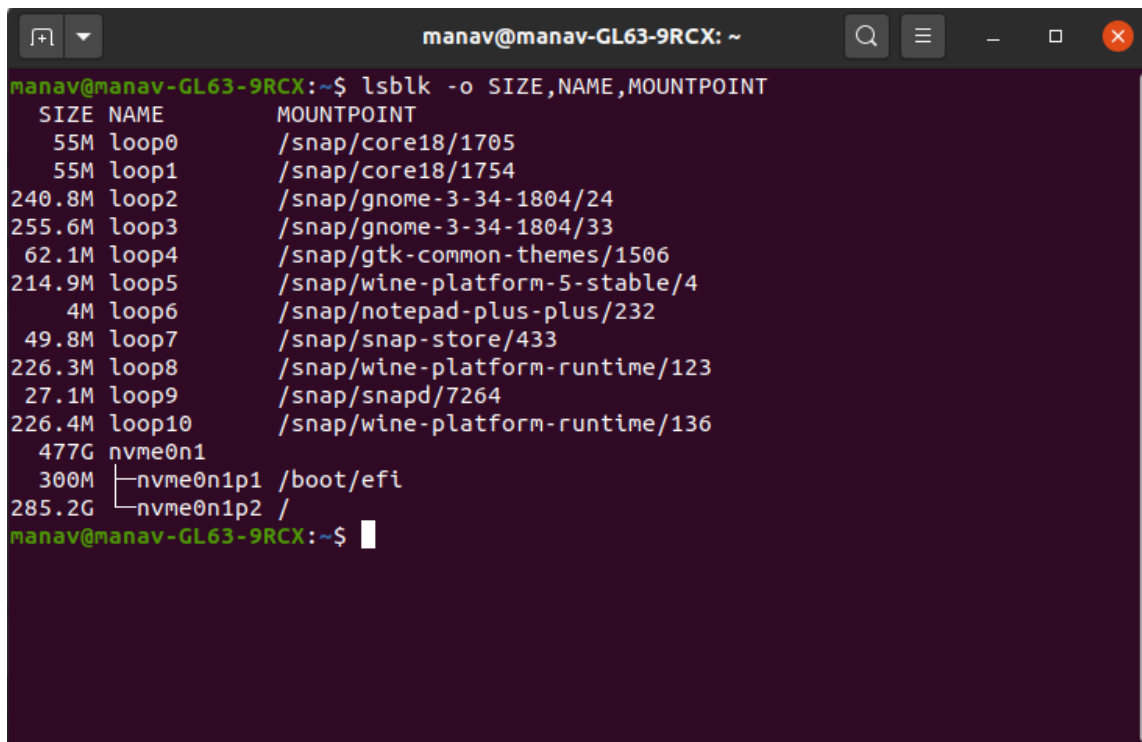


```
manav@manav-GL63-9RCX:~$ lsblk -m
\NAME          SIZE OWNER GROUP MODE
loop0          55M root  disk brw-rw----
loop1          55M root  disk brw-rw----
loop2         240.8M root  disk brw-rw----
loop3         255.6M root  disk brw-rw----
loop4          62.1M root  disk brw-rw----
loop5         214.9M root  disk brw-rw----
loop6           4M root  disk brw-rw----
loop7          49.8M root  disk brw-rw----
loop8         226.3M root  disk brw-rw----
loop9          27.1M root  disk brw-rw----
loop10        226.4M root  disk brw-rw----
nvme0n1        477G root  disk brw-rw----
├─nvme0n1p1    300M root  disk brw-rw----
└─nvme0n1p2  285.2G root  disk brw-rw----
manav@manav-GL63-9RCX:~$
```

This will display all details of device owner, group and mode of block devices.

8. To print selected coulumns of block-devices.

\$lsblk -o SIZE, NAME, MOUNTPOINT

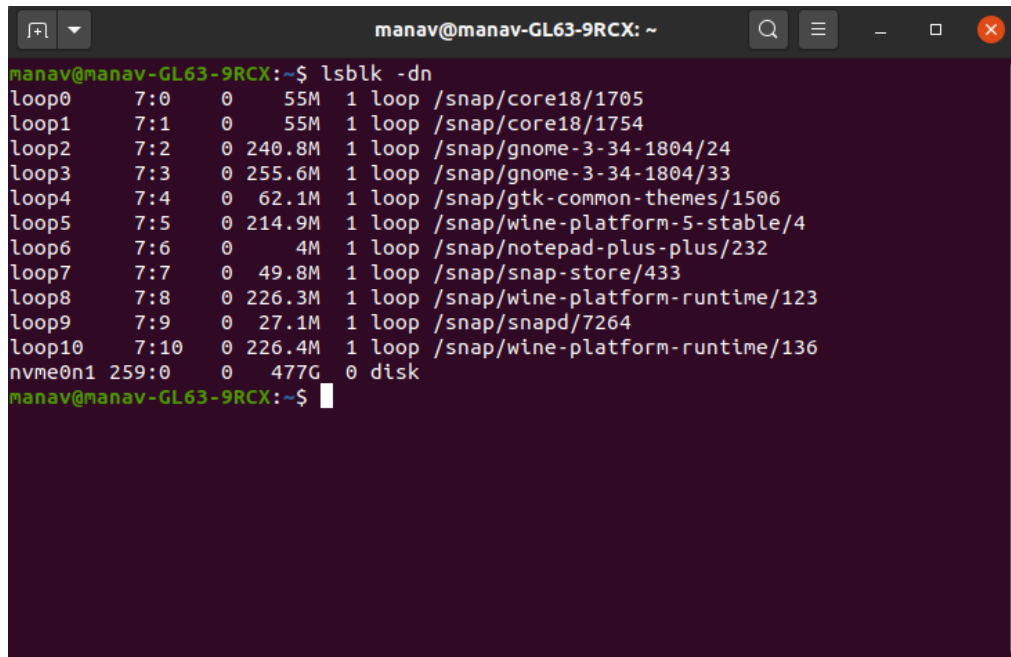


```
manav@manav-GL63-9RCX:~$ lsblk -o SIZE,NAME,MOUNTPOINT
SIZE NAME          MOUNTPOINT
55M loop0          /snap/core18/1705
55M loop1          /snap/core18/1754
240.8M loop2       /snap/gnome-3-34-1804/24
255.6M loop3       /snap/gnome-3-34-1804/33
62.1M loop4        /snap/gtk-common-themes/1506
214.9M loop5       /snap/wine-platform-5-stable/4
4M loop6           /snap/notepad-plus-plus/232
49.8M loop7        /snap/snap-store/433
226.3M loop8       /snap/wine-platform-runtime/123
27.1M loop9        /snap/snapsd/7264
226.4M loop10      /snap/wine-platform-runtime/136
477G nvme0n1
300M └─nvme0n1p1 /boot/efi
285.2G └─nvme0n1p2 /
manav@manav-GL63-9RCX:~$
```

This will print only the specified columns.

9. To hide column headings.

\$lsblk -dn



A terminal window titled "manav@manav-GL63-9RCX: ~" showing the output of the command "lsblk -dn". The output lists various loop and disk devices with their attributes. The terminal has a dark purple background and a light green cursor.

```
manav@manav-GL63-9RCX:~$ lsblk -dn
loop0      7:0      0    55M    1 loop /snap/core18/1705
loop1      7:1      0    55M    1 loop /snap/core18/1754
loop2      7:2      0 240.8M    1 loop /snap/gnome-3-34-1804/24
loop3      7:3      0 255.6M    1 loop /snap/gnome-3-34-1804/33
loop4      7:4      0   62.1M    1 loop /snap/gtk-common-themes/1506
loop5      7:5      0 214.9M    1 loop /snap/wine-platform-5-stable/4
loop6      7:6      0     4M    1 loop /snap/notepad-plus-plus/232
loop7      7:7      0   49.8M    1 loop /snap/snap-store/433
loop8      7:8      0 226.3M    1 loop /snap/wine-platform-runtime/123
loop9      7:9      0   27.1M    1 loop /snap/snapd/7264
loop10     7:10     0 226.4M    1 loop /snap/wine-platform-runtime/136
nvme0n1    259:0     0   477G    0 disk
```

This will hide column headings.

10. To display help section of the command.

\$lsblk --help

```
manav@manav-GL63-9RCX:~$ lsblk --help

Usage:
lsblk [options] [<device> ...]

List information about block devices.

Options:
-D, --discard          print discard capabilities
-E, --dedup <column>  de-duplicate output by <column>
-I, --include <list>  show only devices with specified major numbers
-J, --json             use JSON output format
-O, --output-all      output all columns
-P, --pairs            use key="value" output format
-S, --scsi            output info about SCSI devices
-T, --tree[=<column>] use tree format output
-a, --all             print all devices
-b, --bytes           print SIZE in bytes rather than in human readable format
-d, --nodeps          don't print slaves or holders
-e, --exclude <list>  exclude devices by major number (default: RAM disks)
-f, --fs              output info about filesystems
-i, --ascii           use ascii characters only
-l, --list            use list format output
-M, --merge           group parents of sub-trees (usable for RAID's, Multi-path)
-m, --perms           output info about permissions
-n, --noheadings      don't print headings
-o, --output <list>   output columns
-p, --paths           print complete device path
-r, --raw            use raw output format
-s, --inverse         inverse dependencies
-t, --topology        output info about topology
-z, --zoned           print zone model
-x, --sort <column>   sort output by <column>
    --sysroot <dir>  use specified directory as system root

-h, --help            display this help
-V, --version         display version

Available output columns:
NAME    device name
KNAME   internal kernel device name
PATH    path to the device node
MAJ:MIN major:minor device number
FSAVAIL filesystem size available
FSSIZE  filesystem size
FSTYPE  filesystem type
FSUSED  filesystem size used
FSUSE%  filesystem use percentage
MOUNTPPOINT where the device is mounted
LABEL   filesystem LABEL
UUID    filesystem UUID
PTUUID  partition table identifier (usually UUID)
PTTYPE  partition table type
PARTTYPE partition type UUID
PARTLABEL partition LABEL
PARTUUID partition UUID
```

```

Available output columns:
    NAME    device name
    KNAME    internal kernel device name
    PATH     path to the device node
    MAJ:MIN  major:minor device number
    FSAVAIL  filesystem size available
    FSSIZE   filesystem size
    FSTYPE   filesystem type
    FSUSED   filesystem size used
    FSUSE%   filesystem use percentage
    MOUNTPOINT where the device is mounted
    LABEL    filesystem LABEL
    UUID     filesystem UUID
    PTUUID   partition table identifier (usually UUID)
    PTTYPE   partition table type
    PARTTYPE  partition type UUID
    PARTLABEL partition LABEL
    PARTUUID  partition UUID
    PARTFLAGS partition flags
    RA       read-ahead of the device
    RO       read-only device
    RM       removable device
    HOTPLUG  removable or hotplug device (usb, pcmcia, ...)
    MODEL    device identifier
    SERIAL   disk serial number
    SIZE     size of the device
    STATE    state of the device
    OWNER    user name
    GROUP    group name
    MODE     device node permissions
    ALIGNMENT alignment offset
    MIN-IO   minimum I/O size
    OPT-IO   optimal I/O size
    PHY-SEC  physical sector size
    LOG-SEC  logical sector size
    ROTA     rotational device
    SCHED    I/O scheduler name
    RQ-SIZE  request queue size
    TYPE     device type
    DISC-ALN discard alignment offset
    DISC-GRAN discard granularity
    DISC-MAX discard max bytes
    DISC-ZERO discard zeroes data
    WSAFE    write same max bytes
    WWN      unique storage identifier
    RAND     adds randomness
    PKNAME   internal parent kernel device name
    HCTL     Host:Channel:Target:Lun for SCSI
    TRAN     device transport type
    SUBSYSTEMS de-duplicated chain of subsystems
    REV      device revision
    VENDOR   device vendor
    ZONED    zone model

For more details see lsblk(8).
manav@manav-GL63-9RCX:~$

```

This will display the help section of the command.

E. Linux commands for disk management: sfdisk,

On Unix-like operating systems, the sfdisk command is a **partition table editor**.

It is similar to fdisk and cfdisk, but with additional features. Additionally, unlike those two programs, sfdisk can run non-interactively.

sfdisk has four (main) uses: it can list the size of a partition, list the partitions on a device, check the partitions on a device, and - this one is very dangerous! - it can repartition a device.

sfdisk doesn't understand the GUID Partition Table (GPT) format and it is not designed for large partitions. In these cases use the more advanced GNU parted.

1. Listing partition sizes

The following command:

```
sfdisk -s partition
```

gives the size of partition in blocks. This may be useful in connection with programs like mkswap. Here partition is usually something like /dev/hda1 or /dev/sdb12, but may also be an entire disk, like /dev/xda. The following example command, and output, shows the size of partition /dev/hda9:

```
sfdisk -s /dev/hda9  
81599
```

If the partition argument is omitted, sfdisk will list the sizes of all disks, and the total:

```
sfdisk -s  
/dev/hda: 208896  
/dev/hdb: 1025136  
/dev/hdc: 1031063  
/dev/sda: 8877895  
/dev/sdb: 1758927  
total: 12901917 blocks
```

2. Listing partitions

The second type of invocation:

```
sfdisk -l device
```

will list the partitions on the specified device. If the device argument is omitted, the partitions on all hard disks are listed. The following example lists the partitions on device /dev/hdc:

```
sfdisk -l /dev/hdc
```

Disk /dev/hdc: 16 heads, 63 sectors, 2045 cylinders

Units = cylinders of 516096 bytes, blocks of 1024 bytes, counting from 0

	Device	Boot	Start	End	#cyls	#blocks	Id	System
/dev/hdc1		0+	406	407-	205096+	83	Linux	native
/dev/hdc2		407	813	407	205128	83	Linux	native
/dev/hdc3		814	2044	1231	620424	83	Linux	native
/dev/hdc4		0	-	0	0	0	Empty	

The trailing - and + signs indicate that rounding has taken place, and that the actual value is slightly less or more. To see the exact values, ask for a listing with sectors as the unit (using the "-u S" option).

3. Checking partitions

The third type of invocation:

```
sfdisk -V device
```

will apply various consistency checks to the partition tables on device. It either prints "OK", or complains. The -V option can be used together with -l. For example, in a shell script you could use

```
sfdisk -V -q device
```

which would only return the status of device.

4. Creating partitions

The fourth type of invocation:

```
sfdisk device
```

will cause sfdisk to read the specification for the desired partitioning of device from standard input, and then to change the partition tables on that disk. Thus it is possible **to use sfdisk from a shell script. When sfdisk determines that its standard input is a terminal, it will be conversational; otherwise it will abort on any error.**

BE EXTREMELY CAREFUL. ONE TYPING MISTAKE AND ALL YOUR DATA IS LOST!

As a precaution, you can save the sectors changed by sfdisk:

```
% sfdisk /dev/hdd -O hdd-partition-sectors.save
```

Then, if you discover that you did something stupid before anything else has been written to disk, it may be possible to recover the old situation with:

```
% sfdisk /dev/hdd -I hdd-partition-sectors.save
```

This command is not the same as saving the old partition table: a readable version of the old partition table can be saved using the -d option. However, if you create logical partitions, the sectors describing them are located somewhere on disk, possibly on sectors that were not part of the partition table before. Thus, the information the -O option saves is not a binary version of the output of -d.

Syntax

```
sfdisk [options] device
```

```
sfdisk -s [partition]
```

Options

-v, --version

-h, --help

-T, --list-types

-s, --show-size

-g, --show-geometry

-G, --show-pt-geometry

-l, --list

-d, --dump

-V, --verify

-i, --increment

-N <i>number</i>

-A, --activate <i>number</i>

-c, --id <i>number</i> [<i>Id</i>]

-u, --unit <i>letter</i>

-x, --show-extended

-C,

-O <i>file</i>

-I <i>file</i>

--cylinders <i>cylinders</i>

-H, --heads <i>heads</i>

-S, --sectors <i>sectors</i>

-f, --force

-q, --quiet

-L, --Linux

-D, --DOS

-E, --DOS-extended

--IBM, --leave-last

-n

-R, --re-read

--no-reread

--in-order

--not-in-order

--inside-outer

--not-inside-outer

--nested

--chained

--onesector

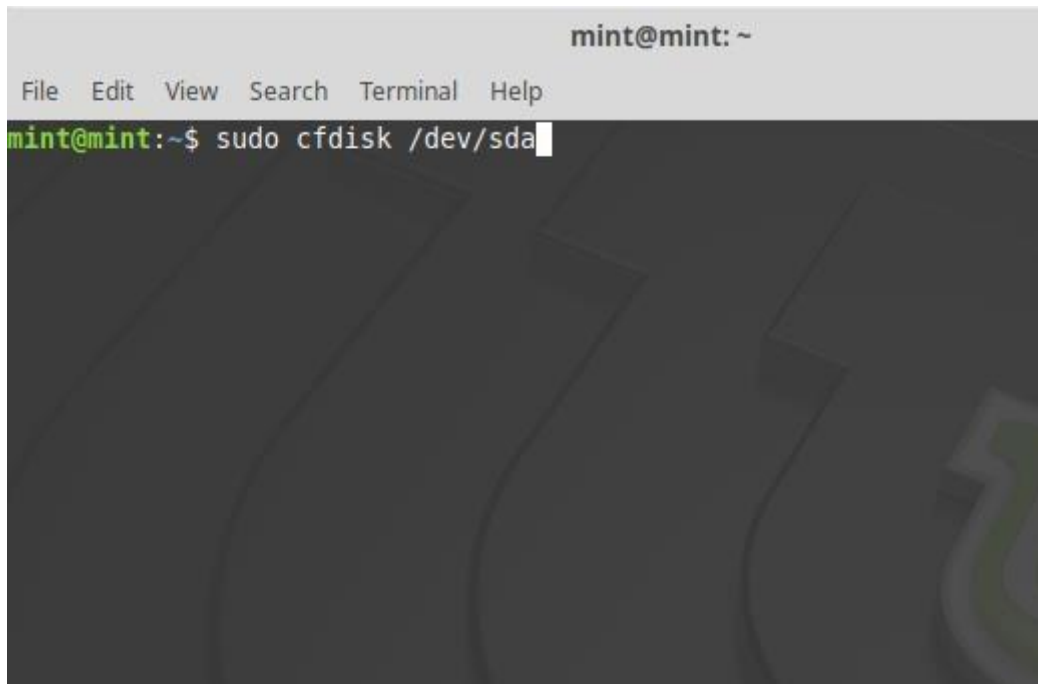
F. Linux commands for disk management: cfdisk

cfdisk command is used to create, delete, and modify partitions on a disk device.

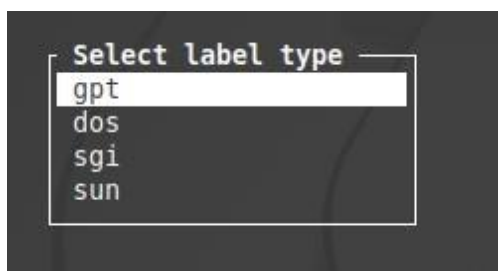
It displays or manipulates the disk partition table by providing a text-based –graphical interface.

cfdisk /dev/sda

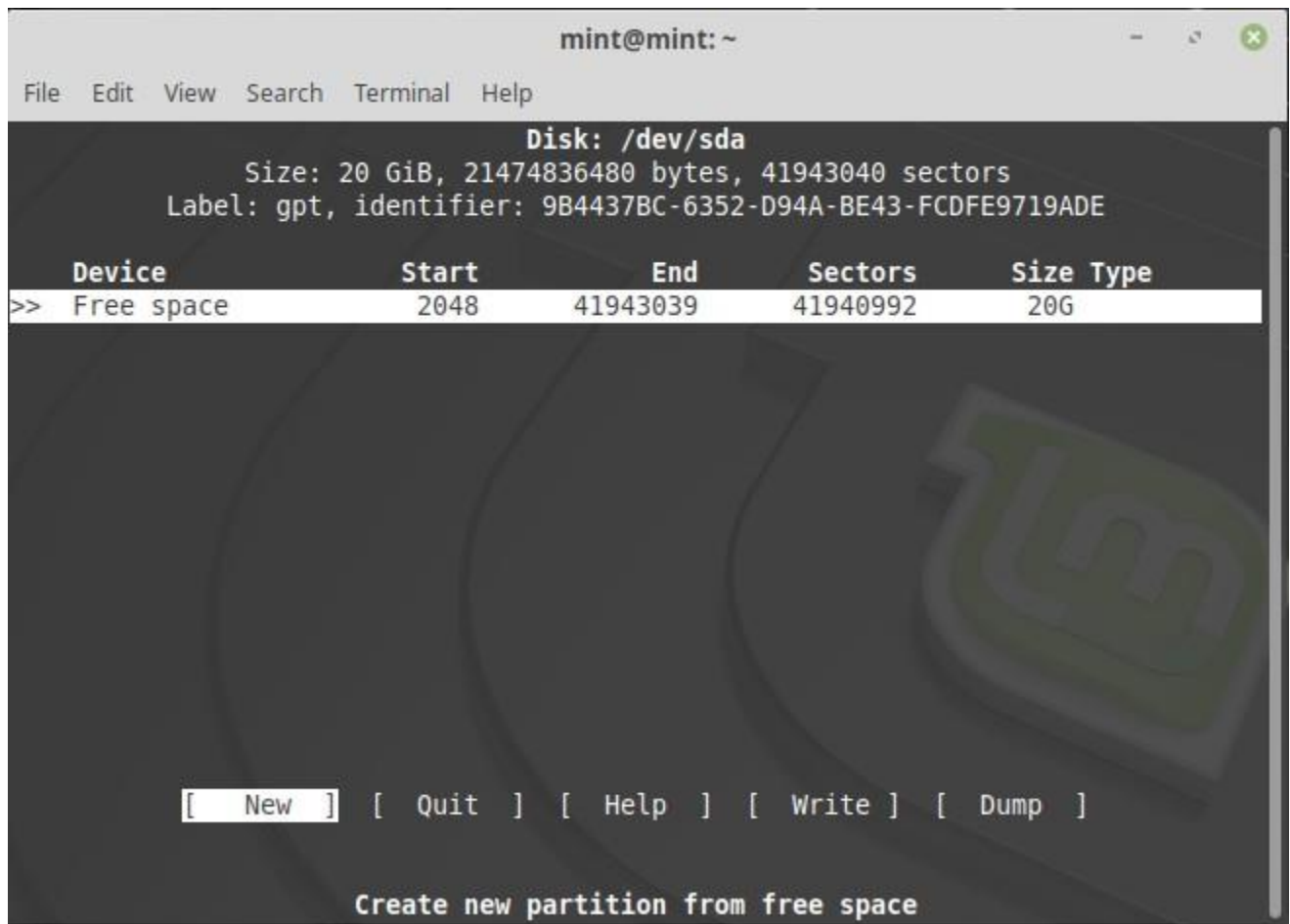
Example:



After running you get a prompt like this:



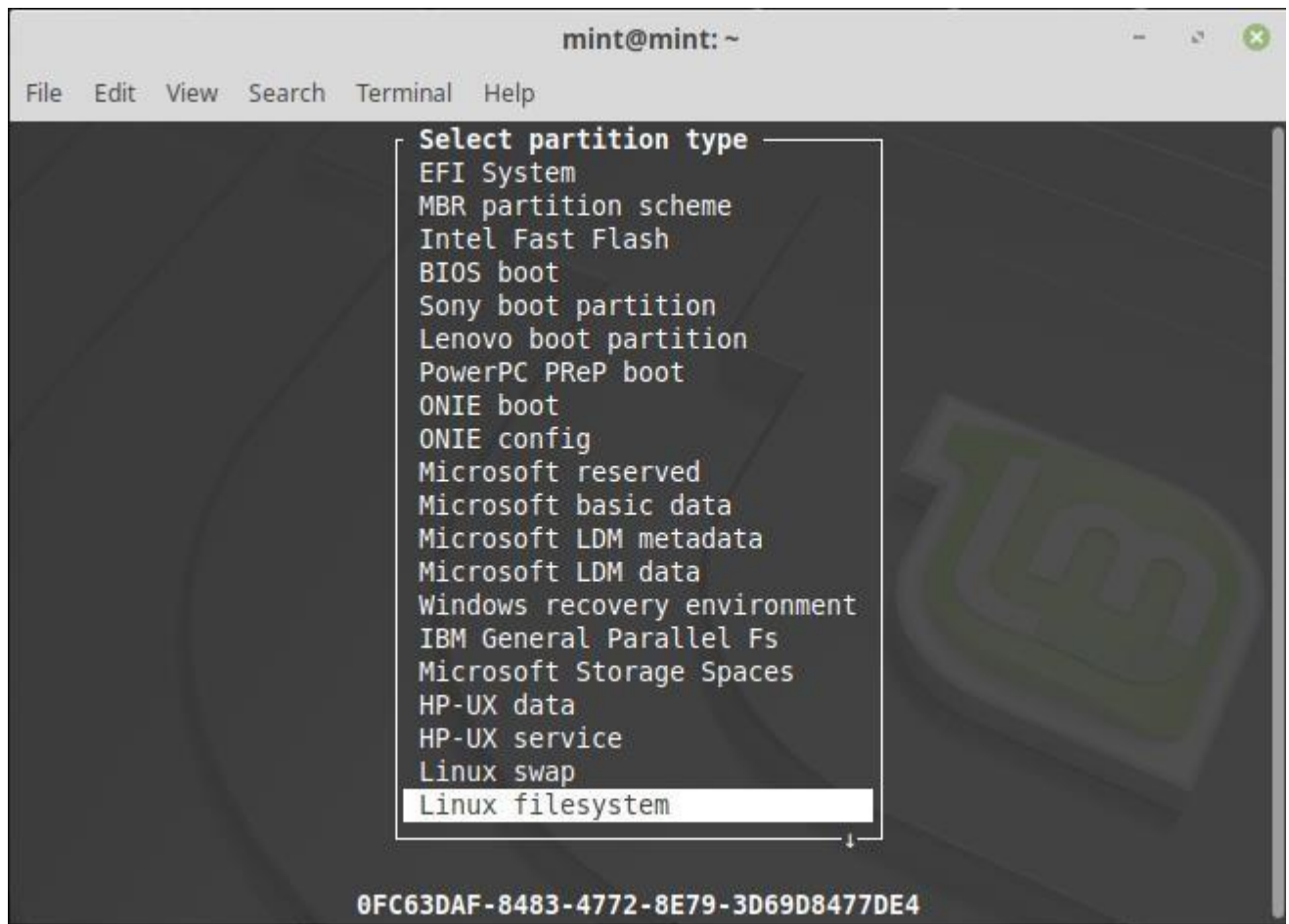
Choose *gpt* from the list. Now you will see a partition table like this:



Creating Partitions Using cfdisk:

- See the available free space. Here we have 20 GB. Select NEW and create a new partition. Use up-down arrow keys to navigate and enter to select.
- You can do many things with the free space, if you are installing a new system with a command line interface, you can see an option of using the selected space as primary partition.

Example: Select the size 2GB. Enter -> and select primary. Similarly we can do a logical partition also.



- After sizing the partition, select what type do you want, in my case, I am choosing Linux Swap.
- After selecting the size and type write to the disk:

```
mint@mint: ~  
File Edit View Search Terminal Help  
Disk: /dev/sda  
Size: 20 GiB, 21474836480 bytes, 41943040 sectors  
Label: gpt, identifier: D799AA21-2B45-DD40-8661-810F22160C03  


| Device       | Start   | End      | Sectors  | Size  | Type       |
|--------------|---------|----------|----------|-------|------------|
| >> /dev/sda1 | 2048    | 3908297  | 3906250  | 1.9G  | Linux swap |
| Free space   | 3909632 | 41943039 | 38033408 | 18.1G |            |



Partition UUID: 0BB83BBC-9C63-5449-B82E-4C9F25FC1290  
Partition type: Linux swap (0657FD6D-A4AB-43C4-84E5-0933C84B4F4F)



[ Delete ] [ Resize ] [ Quit ] [ Type ] [ Help ] [ Write ]



[ Dump ]



Write partition table to disk (this might destroy data)


```

You will see a prompt like this:

```
mint@mint: ~
File Edit View Search Terminal Help

Disk: /dev/sda
Size: 20 GiB, 21474836480 bytes, 41943040 sectors
Label: gpt, identifier: D799AA21-2B45-DD40-8661-810F22160C03

Device      Start      End        Sectors    Size Type
>> /dev/sda1  2048      3908297    3906250    1.9G Linux swap
Free space  3909632   41943039   38033408   18.1G

Partition UUID: 0BB83BBC-9C63-5449-B82E-4C9F25FC1290
Partition type: Linux swap (0657FD6D-A4AB-43C4-84E5-0933C84B4F4F)

Are you sure you want to write the partition table to disk? Y

Type "yes" or "no", or press ESC to leave this dialog.
```

Options:

- -h, --help: It displays help text and exit.
- -L, --color[=when]: Colorize the output. The optional argument *when* can be auto, never or always. If the *when* argument is omitted, it defaults to auto. The colors can be disabled, for the current built-in default see --help output. See also the COLORS section.
- -V, --version: Display version information and exit.
- -z, --zero: Start with an in-memory zeroed partition table. This option does not zero the partition table on the disk; rather, it simply starts the program without reading the existing partition table. This option allows you to create a new partition table from scratch or from a sfdisk-compatible script.

Other command line commands: While using cfdisk you can use simple commands just like we use in vi editor for saving, inserting etc. The list of commands are as follows:

- b: Toggle bootable flag of the current partition. It allows the user to select which partition is primary in the bootable drive. Just press b to see the results, no need of using ctrl.
- d: It will delete the current marked partition, making a free space for new partition.
- h: Will print the help screen, showing commands used .
- n: Will create a new partition of the marked free space .
- q: Will quit the program without writing partitions to the table.
- s: Will fix the partitions order if they are now in proper array.
- t: Will allow you to change the partition type, allowing you to select from the list.
- u: Will dump the disk layout in a specified script file name
- W: Will allow the user to write the data to the disk. The user will be asked if he or she wants to write or not by simply taking input -yes or -no.
- x: Will allow the user to hide or display all extra information of the partition.
- Up-Arrow: Will allow the user to move the cursor to the previous partition, like moving up in the given table list.
- Down-Arrow: This option allows the user to move the cursor to the next partition, next partition because every new partition is placed after the previous partition.
- Left-Arrow: This option allows the user to enter previous menu item.
- Right-Arrow: This option allows the user to enter to the next menu item.

Example: Sample output when we use `-?ll` or `-hll`

```
This is cfdisk, a curses-based disk partitioning program.
It lets you create, delete, and modify partitions on a block device.

Command      Meaning
-----
b            Toggle bootable flag of the current partition
d            Delete the current partition
h            Print this screen
n            Create new partition from free space
q            Quit program without writing partition table
s            Fix partitions order (only when in disarray)
t            Change the partition type
u            Dump disk layout to sfdisk compatible script file
W            Write partition table to disk (you must enter uppercase W);
              since this might destroy data on the disk, you must either
              confirm or deny the write by entering 'yes' or 'no'
x            Display/hide extra information about a partition
Up Arrow     Move cursor to the previous partition
Down Arrow   Move cursor to the next partition
Left Arrow   Move cursor to the previous menu item
Right Arrow  Move cursor to the next menu item

Note: All of the commands can be entered with either upper or lower
case letters (except for Write).

Use lsblk(8) or partx(8) to see more details about the device.

Copyright (C) 2014-2017 Karel Zak <kzak@redhat.com>

Press a key to continue.
```


5.4 Creation and deletion of partition in Linux:

Create Partitions in Linux:

Creating disk partitions enables you to split your hard drive into multiple sections that act independently.

In Linux, users must structure storage devices (USB and hard drives) before using them. Partitioning is also useful when you are installing multiple operating systems on a single machine.

Option 1: Partition a Disk Using parted Command:

Follow the steps below to partition a disk in Linux by using the **parted** command.

Step 1: List Partitions

Before making a partition, list available storage devices and partitions. This action helps identify the storage device you want to partition.

Run the following command with **sudo** to list storage devices and partitions:

```
sudo parted -l
```

The terminal prints out available storage devices with information about:

- **Model** – Model of the storage device.
- **Disk** – Name and size of the disk.
- **Sector size** – Logical and physical size of the memory. Not to be confused with [available disk space](#).
- **Partition Table** – Partition table type (msdos, gpt, aix, amiga, bsd, dvh, mac, pc98, sun, and loop).
- **Disk Flags** – Partitions with information on size, type, file system, and flags.

Partitions types can be:

- **Primary** – Holds the operating system files. Only four primary partitions can be created.
- **Extended** – Special type of partition in which more than the four primary partitions can be created.
- **Logical** – Partition that has been created inside of an extended partition.

For example, there are two storage devices (/dev/sda and /dev/sdb):

```
nevena@nevena-VirtualBox:~$ sudo parted -l
[sudo] password for nevena:
Model: ATA VBOX HARDDISK (scsi)
Disk /dev/sda: 33,3GB
Sector size (logical/physical): 512B/512B
Partition Table: msdos
Disk Flags:

Number  Start   End     Size    Type     File system  Flags
  1      1049kB  538MB   537MB   primary  fat32        boot
  2      539MB   33,3GB  32,8GB   extended
  5      539MB   33,3GB  32,8GB   logical  ext4

Model: ATA VBOX HARDDISK (scsi)
Disk /dev/sdb: 10,6GB
Sector size (logical/physical): 512B/512B
Partition Table: gpt
Disk Flags:

Number  Start   End     Size    File system  Name     Flags
  1      17,4kB  1396MB  1396MB                   primary
```

Note: The first storage disk (dev/sda or dev/vda) contains the operating system. Creating a partition on this disk can make your system unbootable. Only create partitions on secondary disks (dev/sdb, dev/sdc, dev/vdb, or dev/vdc).

Step 2: Open Storage Disk

Open the storage disk that you intend to partition by running the following command:

`sudo parted /dev/sdb`

```
nevena@nevena-VirtualBox:~$ sudo parted /dev/sdb
GNU Parted 3.3
Using /dev/sdb
Welcome to GNU Parted! Type 'help' to view a list of commands.
(parted) █
```

Always specify the storage device. If you don't specify a disk name, the disk is randomly selected. To change the disk to **dev/sdb** run:

`select /dev/sdb`

The **dev/sdb** disk is open:

```
nevena@nevena-VirtualBox:~$ sudo parted
GNU Parted 3.3
Using /dev/sda
Welcome to GNU Parted! Type 'help' to view a list of commands.
(parted) select /dev/sdb
Using /dev/sdb
(parted) █
```

Step 3: Make a Partition Table

Create a partition table before partitioning the disk. A partition table is located at the start of a hard drive and it stores data about the size and location of each partition.

Partition table types are: **aix**, **amiga**, **bsd**, **dvh**, **gpt**, **mac**, **ms-dos**, **pc98**, **sun**, and **loop**.

To create a partition table, enter the following:

```
mklabel [partition_table_type]
```

For example, to create a **gpt** partition table, run the following command:

```
mklabel gpt
```

Type **Yes** to execute:

```
(parted) mklabel gpt
Warning: The existing disk label on /dev/sdb will be destroyed and all data on this
disk will be lost. Do you want to continue?
Yes/No? █
```

Note: The two most commonly used partition table types are **gpt** and **msdos**. The latter supports up to sixteen partitions and formats up to 16TB of space while gpt formats up to 9.4ZB and supports up to 128 partitions.

Step 4: Check Table

Run the **print** command to review the partition table. The output displays information about the storage device:

```
(parted) print
Model: ATA VBOX HARDDISK (scsi)
Disk /dev/sdb: 10,6GB
Sector size (logical/physical): 512B/512B
Partition Table: gpt
Disk Flags:

Number  Start  End  Size  File system  Name  Flags
(parted) █
```

Note: Run **help mkpart** command to get additional help on how to create a new partition.

Step 5: Create Partition

Let's make a new 1854MB-partition using the ext4 file system. The assigned disk start shall be 1MB and the disk end is at 1855MB.

To create a new partition, enter the following:

```
mkpart primary ext4 1MB 1855MB
```

After that, run the **print** command to review information on the newly created partition. The information is displayed under the *Disk Flags* section:

```
(parted) mkpart primary ext4 1MB 1855MB  
(parted) PRINT  
Model: ATA VBOX HARDDISK (scsi)  
Disk /dev/sdb: 10,6GB  
Sector size (logical/physical): 512B/512B  
Partition Table: gpt  
Disk Flags:  


| Number | Start  | End    | Size   | File system | Name    | Flags |
|--------|--------|--------|--------|-------------|---------|-------|
| 1      | 1049kB | 1855MB | 1854MB | ext4        | primary |       |


```

In a gpt partition table, the partition type is the mandatory partition name. In our example, **primary** is the name of the partition, not the partition type.

To save your actions and quit, enter the **quit** command. Changes are saved automatically with this command.

```
(parted) quit  
Information: You may need to update /etc/fstab.  
nevena@nevena-VirtualBox:~$
```

Note: The “*You may need to update /etc/fstab file*” message signals that the partition can be mounted automatically at boot time.

Option 2: Partition a Disk Using fdisk Command

Follow the steps below to partition a disk in Linux by using the **fdisk** command.

Step 1: List Existing Partitions

Run the following command to list all existing partitions:

```
sudo fdisk -l
```

The output contains information about storage disks and partitions:

```
Disk /dev/sda: 31,3 GiB, 33312931840 bytes, 65064320 sectors
Disk model: VBOX HARDDISK
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x558d572e
```

Device	Boot	Start	End	Sectors	Size	Id	Type
/dev/sda1	*	2048	1050623	1048576	512M	b	W95 FAT32
/dev/sda2		1052670	65062911	64010242	30,5G	5	Extended
/dev/sda5		1052672	65062911	64010240	30,5G	83	Linux

```
Disk /dev/sdb: 9,91 GiB, 10621960192 bytes, 20746016 sectors
Disk model: VBOX HARDDISK
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: gpt
Disk identifier: 96200583-C460-44DD-A69A-7B376C533B5D
```

Device	Start	End	Sectors	Size	Type
/dev/sdb1	2048	3622911	3620864	1,7G	Linux filesystem

Step 2: Select Storage Disk

Select the storage disk you want to create partitions on by running the following command:

```
sudo fdisk /dev/sdb
```

The **/dev/sdb** storage disk is open:

```
nevena@nevena-VirtualBox:~$ sudo fdisk /dev/sdb

Welcome to fdisk (util-linux 2.34).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.
```

Step 3: Create a New Partition

1. Run the **n** command to create a new partition.
2. Select the partition number by typing the default number (2).
3. After that, you are asked for the starting and ending sector of your hard drive. It is best to type the default number in this section (3622912).
4. The last prompt is related to the size of the partition. You can choose to have several sectors or to set the size in megabytes or gigabytes. Type **+2GB** to set the size of the partition to 2GB.

A message appears confirming that the partition is created.

```
Command (m for help): n
Partition number (2-128, default 2): 2
First sector (3622912-20745982, default 3622912): 3622912
Last sector, +/-sectors or +/-size{K,M,G,T,P} (3622912-20745982, default 20745982): +2G
B
Created a new partition 2 of type 'Linux filesystem' and of size 1,9 GiB.
```

Step 4: Write on Disk

The system created the partition, but the changes are not written on the disk.

1. To write the changes on disk, run the **w** command:

```
Command (m for help): w
The partition table has been altered.
Calling ioctl() to re-read partition table.
Syncing disks.
```

2. Verify that the partition is created by running the following command:

`sudo fdisk -l`

As you can see, the partition **/dev/sdb2** has been created.

```
Disk /dev/sda: 31,3 GiB, 33312931840 bytes, 65064320 sectors
Disk model: VBOX HARDDISK
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x558d572e
```

Device	Boot	Start	End	Sectors	Size	Id	Type
/dev/sda1	*	2048	1050623	1048576	512M	b	W95 FAT32
/dev/sda2		1052670	65062911	64010242	30,5G	5	Extended
/dev/sda5		1052672	65062911	64010240	30,5G	83	Linux

```
Disk /dev/sdb: 9,91 GiB, 10621960192 bytes, 20746016 sectors
Disk model: VBOX HARDDISK
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: gpt
Disk identifier: 96200583-C460-44DD-A69A-7B376C533B5D
```

Device	Start	End	Sectors	Size	Type
/dev/sdb1	2048	3622911	3620864	1,7G	Linux filesystem
/dev/sdb2	3622912	7528447	3905536	1,9G	Linux filesystem

Delete Partition in Linux:

Linux users create partitions to organize their data in an efficient manner. As easy as they are created, Linux partitions can be deleted to reformat a storage device and regain storage space.

Deleting a partition in Linux requires selecting the disk containing the partition and using the **fdisk** command-line utility to delete it.

Follow the steps outlined in the sections below to delete partitions in Linux.

Note: The **fdisk** command-line utility is a text-based partition table manipulator. It is used for partitioning and repartitioning storage devices.

Step 1: List Partition Scheme

Before deleting a partition, run the following command to list the partition scheme.

`fdisk -l`

In our case, the terminal prints out information about two disks: **/dev/sda** and **/dev/sdb**.

Disk **/dev/sda** contains the operating system, therefore its partitions should not be deleted.

```
Disk /dev/sda: 31,3 GiB, 33312931840 bytes, 65064320 sectors
Disk model: VBOX HARDDISK
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x558d572e
```

Device	Boot	Start	End	Sectors	Size	Id	Type
/dev/sda1	*	2048	1050623	1048576	512M	b	W95 FAT32
/dev/sda2		1052670	65062911	64010242	30,5G	5	Extended
/dev/sda5		1052672	65062911	64010240	30,5G	83	Linux

```
Disk /dev/sdb: 9,91 GiB, 10621960192 bytes, 20746016 sectors
Disk model: VBOX HARDDISK
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: gpt
Disk identifier: 96200583-C460-44DD-A69A-7B376C533B5D
```

Device	Start	End	Sectors	Size	Type
/dev/sdb1	2048	3622911	3620864	1,7G	Linux filesystem

```
nevena@nevena-VirtualBox:~$
```

On the **/dev/sdb** disk, there is a **/dev/sdb1** partition which we are about to delete.

Note: The number 1 in **/dev/sdb1** indicates the partition number. Make a note of the number of the partition you intend to delete.

Step 2: Select the Disk

Select the disk that contains the partition you intend to delete.

Common disk names on Linux include:

Type of disk	Disk names	Commonly used disk names
IDE	/dev/hd[a-h]	/dev/hda, /dev/hdb
SCSI	/dev/sd[a-p]	/dev/sda, /dev/sdb
ESDI	/dev/ed[a-d]	/dev/eda
XT	/dev/xd[ab]	/dev/xda

To select a disk, run the following command:

```
sudo fdisk /dev/sdb
```

```
nevena@nevena-VirtualBox:~$ sudo fdisk /dev/sdb

Welcome to fdisk (util-linux 2.34).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

Command (m for help):
```

Step 3: Delete Partitions

Before deleting a partition, back up your data. All data is automatically deleted when a partition is deleted.

To delete partition, run the **d** command in the **fdisk** command-line utility.

The partition is automatically selected if there are no other partitions on the disk. If the disk contains multiple partitions, select a partition by typing its number.

The terminal prints out a message confirming that the partition is deleted.

```
Command (m for help): d
Selected partition 1
Partition 1 has been deleted.

Command (m for help):
```

Note: If you wish to delete multiple partitions, repeat this step as many times as necessary.

Step 4: Verify Partition Deletion

Reload the partition table to verify that the partition has been deleted. To do so, run the **p** command.

The terminal prints out the partition structure of the disk selected in Step 2.


```
Command (m for help): p
Disk /dev/sdb: 9,91 GiB, 10621960192 bytes, 20746016 sectors
Disk model: VBOX HARDDISK
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: gpt
Disk identifier: 96200583-C460-44DD-A69A-7B376C533B5D

Command (m for help):
```

Step 5: Save Changes and Quit

Run the **w** command to write and save changes made to the disk.

```
Command (m for help): w
The partition table has been altered.
Calling ioctl() to re-read partition table.
Syncing disks.

nevena@nevena-VirtualBox:~$
```

Formatting partition :mkfs

Once a partition has been created with the **parted** of **fdisk** command, format it before using it.

Format the partition by running the following command:

```
sudo mkfs -t ext4 /dev/sdb1
```

```
nevena@nevena-VirtualBox:~$ sudo mkfs -t ext4 /dev/sdb1
mke2fs 1.45.5 (07-Jan-2020)
Creating filesystem with 452608 4k blocks and 113344 inodes
Filesystem UUID: d2d0d85b-aac8-4c67-a92a-901779c9ce2d
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912

Allocating group tables: done
Writing inode tables: done
Creating journal (8192 blocks): done
Writing superblocks and filesystem accounting information: done
```

Note: Check out our guide and learn how to format and mount disk partitions in Linux using ext4, FAT32, or NTFS file system!

Mount the Partition

To begin interacting with the disk, create a **mount point** and **mount the partition** to it.

1. Create a mount point by running the following command:

```
sudo mkdir -p /mt/sdb1
```

2. After that, mount the partition by entering:

```
sudo mount -t auto /dev/sdb1 /mt/sdb1
```

The terminal does not print out an output if the commands are executed successfully.

3. Verify if partition is mounted by using the **df hT** command:

```
nevena@nevena-VirtualBox:~$ sudo mkdir -p /mt/sdb1
nevena@nevena-VirtualBox:~$ sudo mount -t auto /dev/sdb1 /mt/sdb1
nevena@nevena-VirtualBox:~$ df -hT
```

Filesystem	Type	Size	Used	Avail	Use%	Mounted on
udev	devtmpfs	1,2G	0	1,2G	0%	/dev
tmpfs	tmpfs	249M	1,4M	248M	1%	/run
/dev/sda5	ext4	30G	7,1G	22G	25%	/
tmpfs	tmpfs	1,3G	0	1,3G	0%	/dev/shm
tmpfs	tmpfs	5,0M	4,0K	5,0M	1%	/run/lock
tmpfs	tmpfs	1,3G	0	1,3G	0%	/sys/fs/cgroup
/dev/loop0	squashfs	55M	55M	0	100%	/snap/core18/1880
/dev/loop1	squashfs	56M	56M	0	100%	/snap/core18/1885
/dev/loop2	squashfs	63M	63M	0	100%	/snap/gtk-common-themes/1506
/dev/loop3	squashfs	256M	256M	0	100%	/snap/gnome-3-34-1804/36
/dev/loop4	squashfs	30M	30M	0	100%	/snap/snapd/8790
/dev/loop5	squashfs	50M	50M	0	100%	/snap/snap-store/467
/dev/loop6	squashfs	31M	31M	0	100%	/snap/snapd/9279
/dev/sda1	vfat	511M	4,0K	511M	1%	/boot/efi
tmpfs	tmpfs	249M	20K	249M	1%	/run/user/1000
/dev/sdb1	ext4	1,7G	5,2M	1,6G	1%	/mt/sdb1

```
nevena@nevena-VirtualBox:~$
```

Note: If you have NTFS partitions on your hard drive, check out our article on how to mount NTFS partitions in Linux.